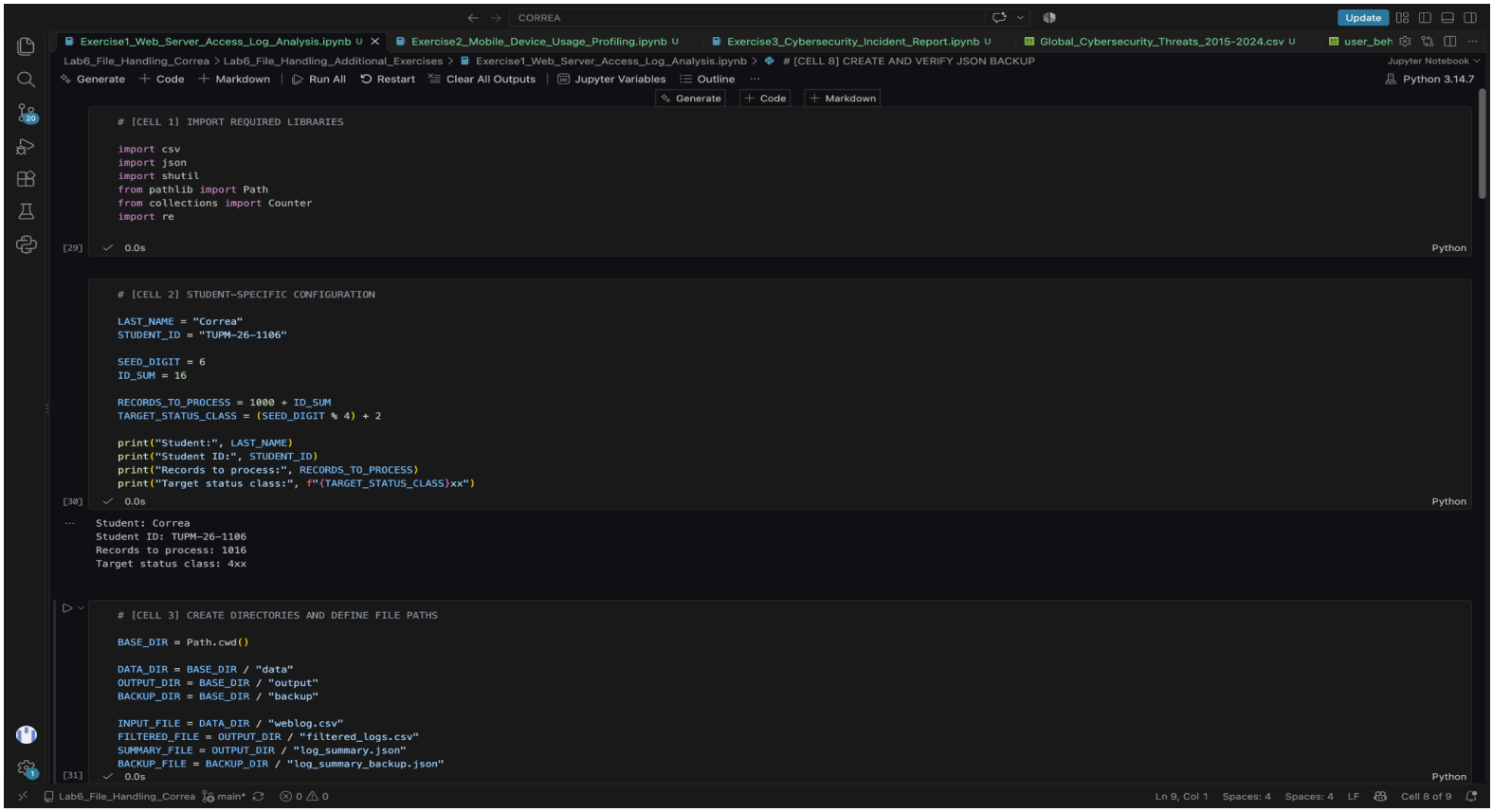
Princess Jherielle M. Correa
BSECE - 1C

November 8, 2026
Computer Programming

Activity 6

FILE HANDLING

EXERCISE #1 WEB SERVER ACCESS LOG ANALYSIS



```
# [CELL 1] IMPORT REQUIRED LIBRARIES

import csv
import json
import shutil
from pathlib import Path
from collections import Counter
import re
```

[29] 0.0s Python

```
# [CELL 2] STUDENT-SPECIFIC CONFIGURATION

LAST_NAME = "Correa"
STUDENT_ID = "TUPM-26-1106"

SEED_DIGIT = 6
ID_SUM = 16

RECORDS_TO_PROCESS = 1000 + ID_SUM
TARGET_STATUS_CLASS = (SEED_DIGIT % 4) + 2

print("Student:", LAST_NAME)
print("Student ID:", STUDENT_ID)
print("Records to process:", RECORDS_TO_PROCESS)
print("Target status class:", f"{TARGET_STATUS_CLASS}xx")
```

[30] 0.0s Python

```
Student: Correa
Student ID: TUPM-26-1106
Records to process: 1016
Target status class: 4xx
```

```
# [CELL 3] CREATE DIRECTORIES AND DEFINE FILE PATHS

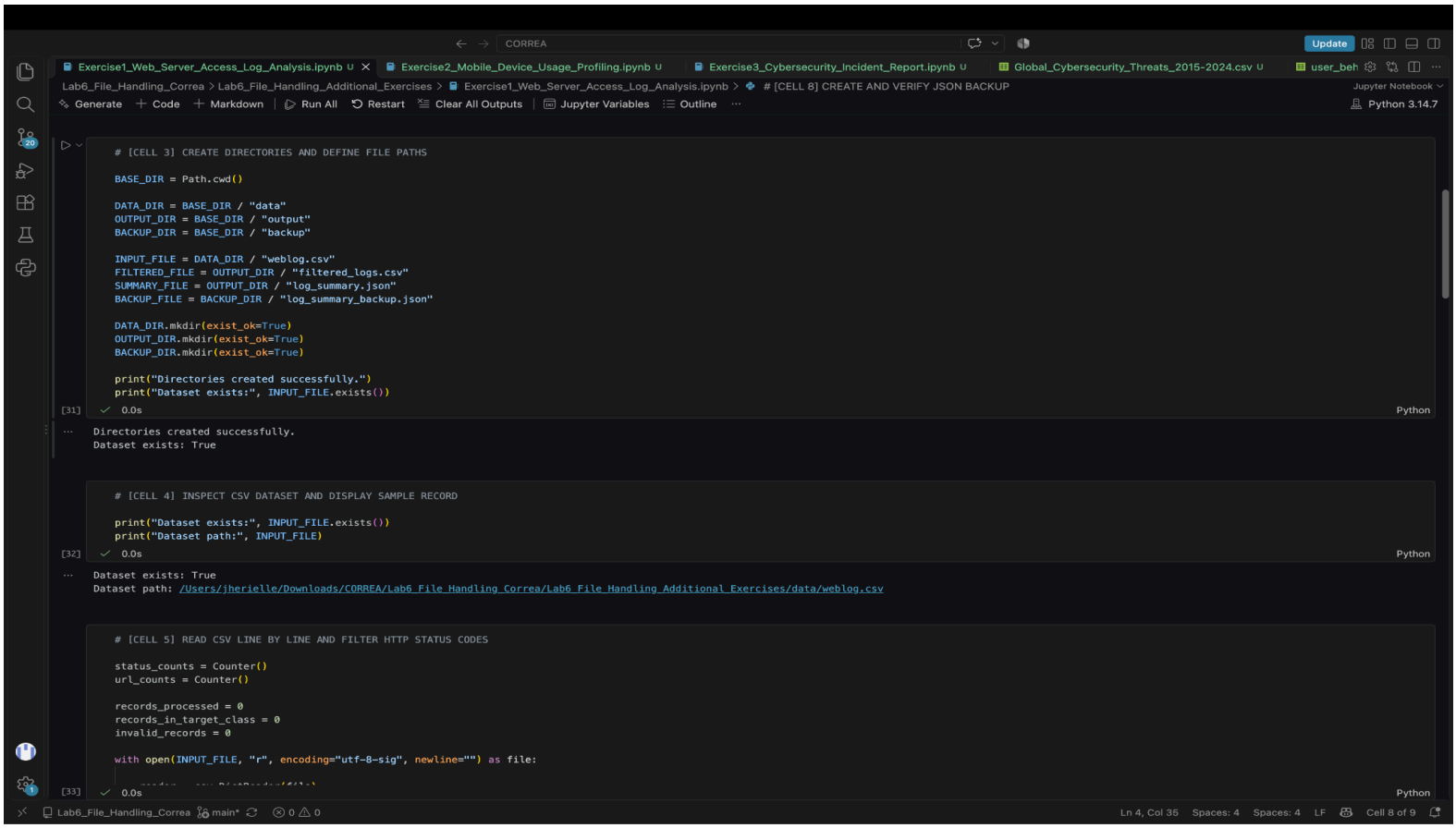
BASE_DIR = Path.cwd()

DATA_DIR = BASE_DIR / "data"
OUTPUT_DIR = BASE_DIR / "output"
BACKUP_DIR = BASE_DIR / "backup"

INPUT_FILE = DATA_DIR / "weblog.csv"
FILTERED_FILE = OUTPUT_DIR / "filtered_logs.csv"
SUMMARY_FILE = OUTPUT_DIR / "log_summary.json"
BACKUP_FILE = BACKUP_DIR / "log_summary_backup.json"
```

[31] 0.0s Python

Lab6_File_Handling_Correa main* 0 0 Ln 9, Col 1 Spaces: 4 Spaces: 4 LF Cell 8 of 9



```
# [CELL 3] CREATE DIRECTORIES AND DEFINE FILE PATHS

BASE_DIR = Path.cwd()

DATA_DIR = BASE_DIR / "data"
OUTPUT_DIR = BASE_DIR / "output"
BACKUP_DIR = BASE_DIR / "backup"

INPUT_FILE = DATA_DIR / "weblog.csv"
FILTERED_FILE = OUTPUT_DIR / "filtered_logs.csv"
SUMMARY_FILE = OUTPUT_DIR / "log_summary.json"
BACKUP_FILE = BACKUP_DIR / "log_summary_backup.json"

DATA_DIR.mkdir(exist_ok=True)
OUTPUT_DIR.mkdir(exist_ok=True)
BACKUP_DIR.mkdir(exist_ok=True)

print("Directories created successfully.")
print("Dataset exists:", INPUT_FILE.exists())
```

[31] 0.0s Python

```
Directories created successfully.
Dataset exists: True
```

```
# [CELL 4] INSPECT CSV DATASET AND DISPLAY SAMPLE RECORD

print("Dataset exists:", INPUT_FILE.exists())
print("Dataset path:", INPUT_FILE)
```

[32] 0.0s Python

```
Dataset exists: True
Dataset path: /Users/jherielle/Downloads/CORREA/Lab6_File_Handling_Correa/Lab6_File_Handling_Additional_Exercises/data/weblog.csv
```

```
# [CELL 5] READ CSV LINE BY LINE AND FILTER HTTP STATUS CODES

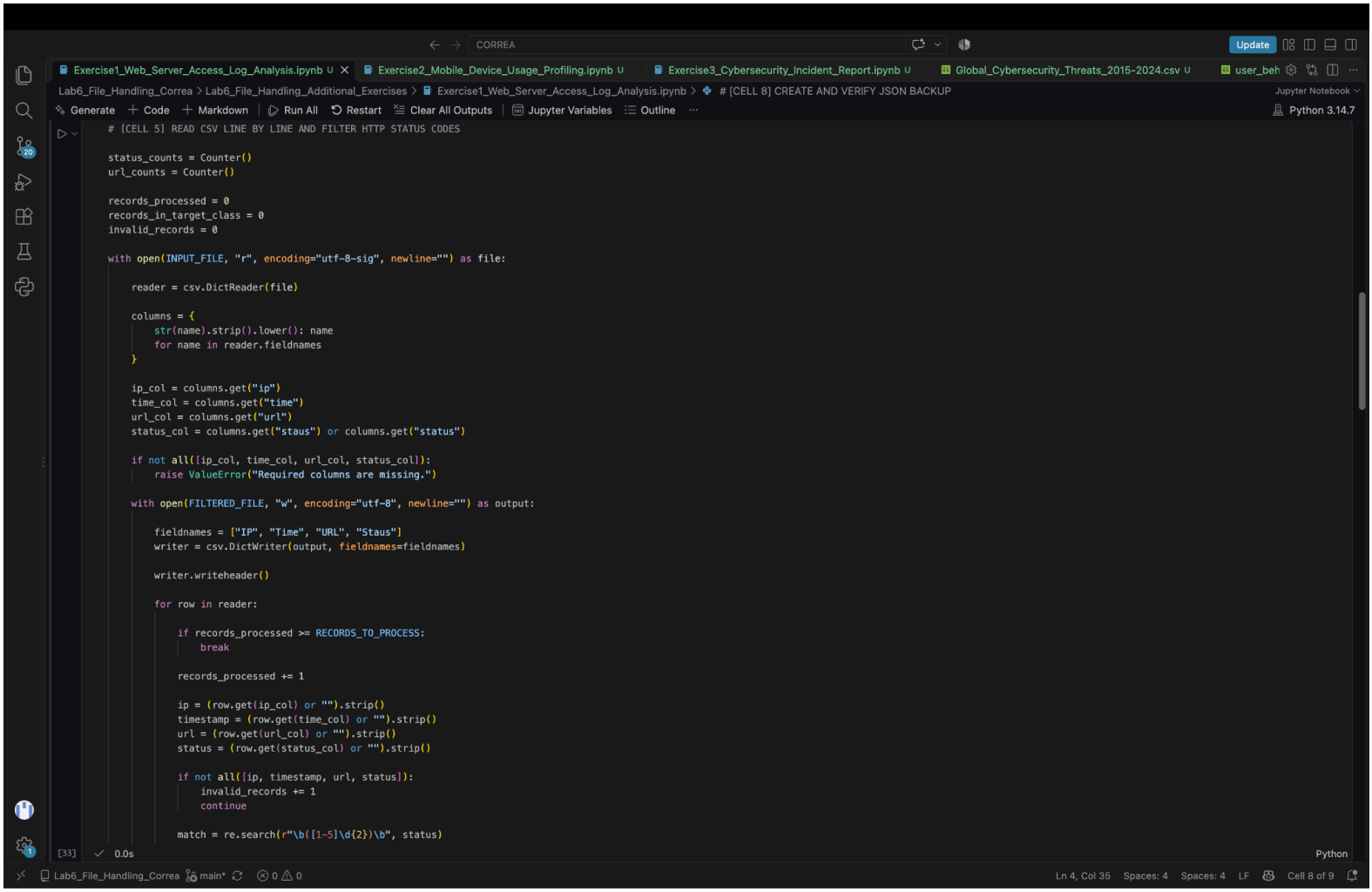
status_counts = Counter()
url_counts = Counter()

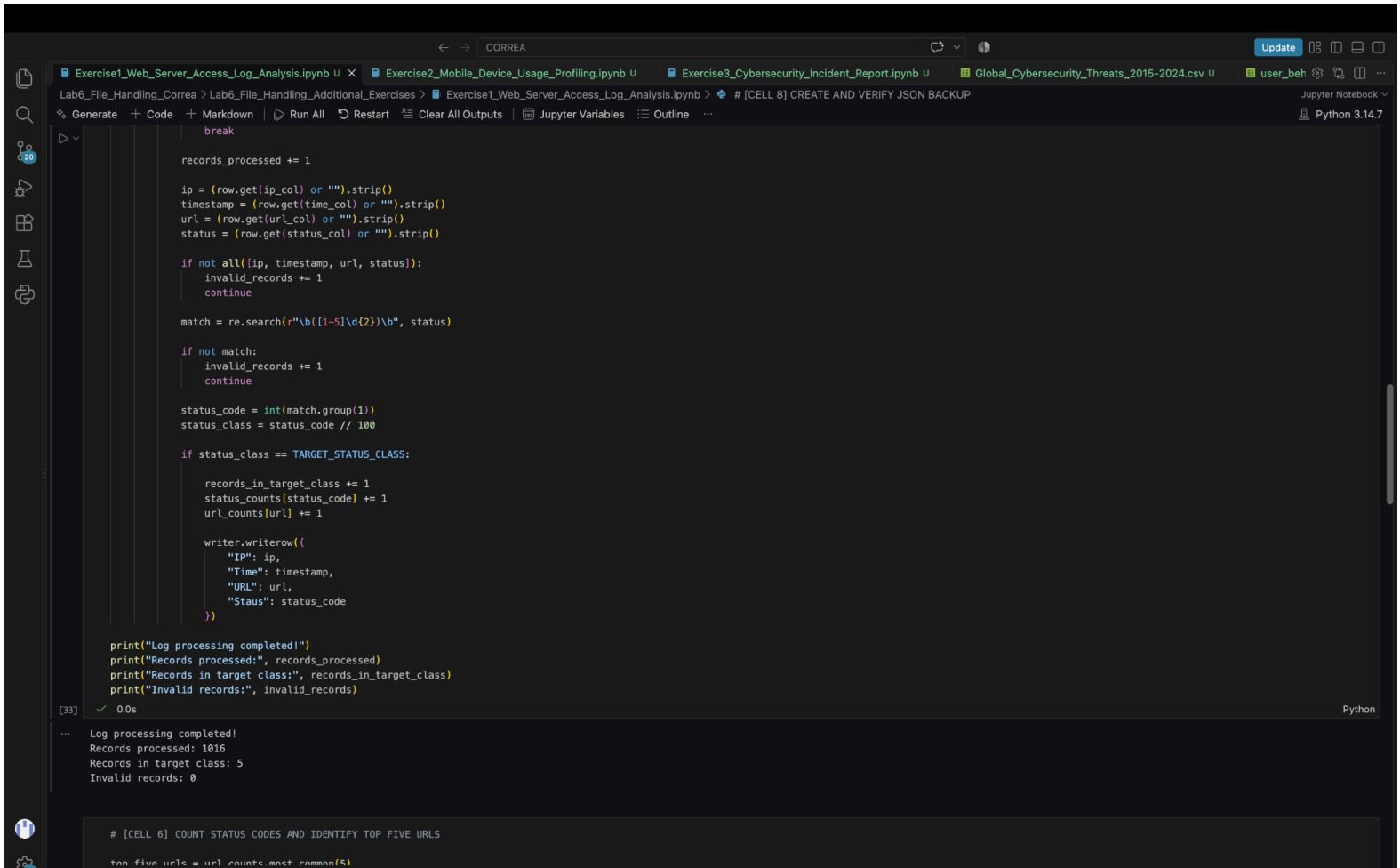
records_processed = 0
records_in_target_class = 0
invalid_records = 0

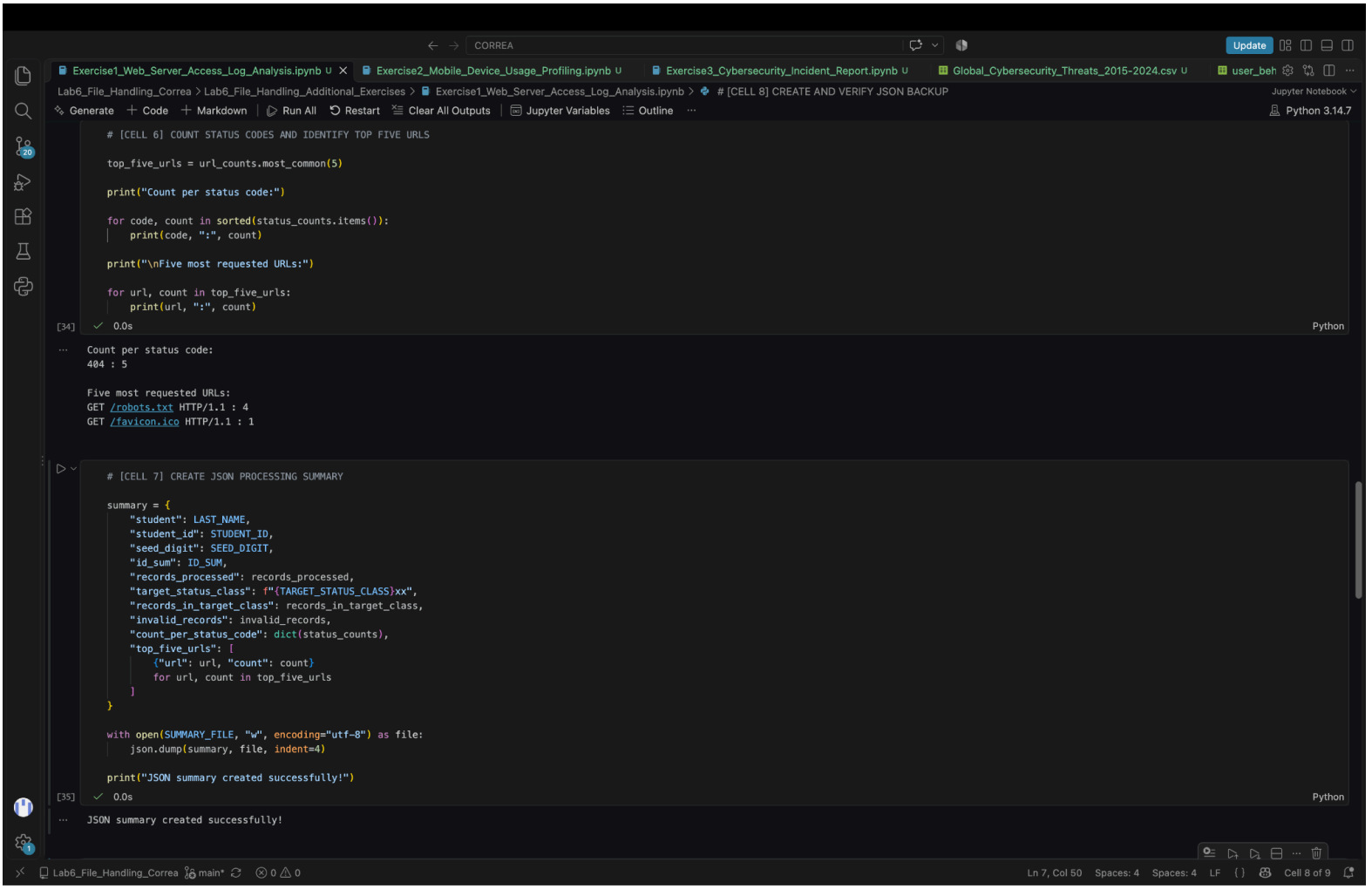
with open(INPUT_FILE, "r", encoding="utf-8-sig", newline="") as file:
```

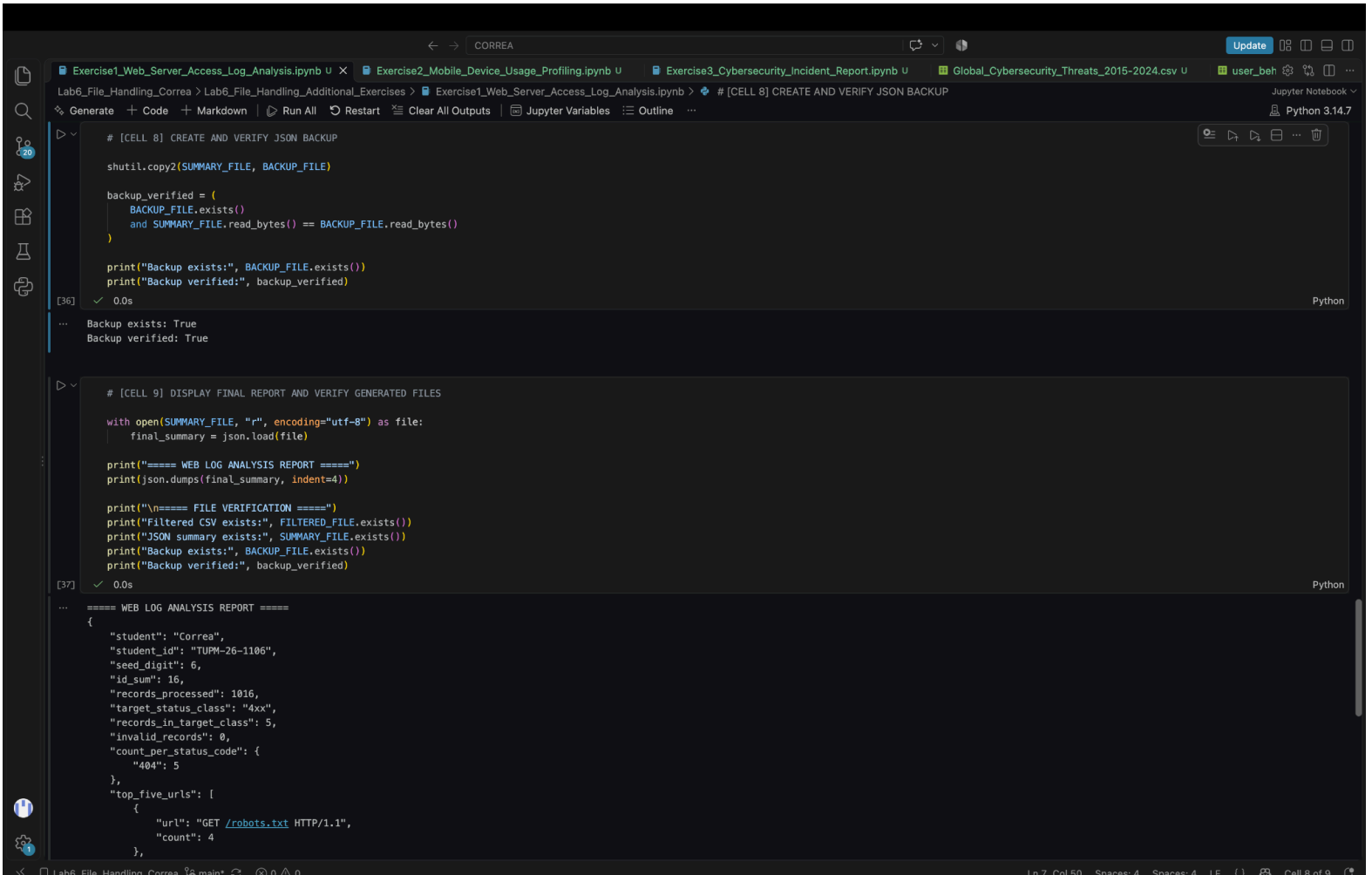
[33] 0.0s Python

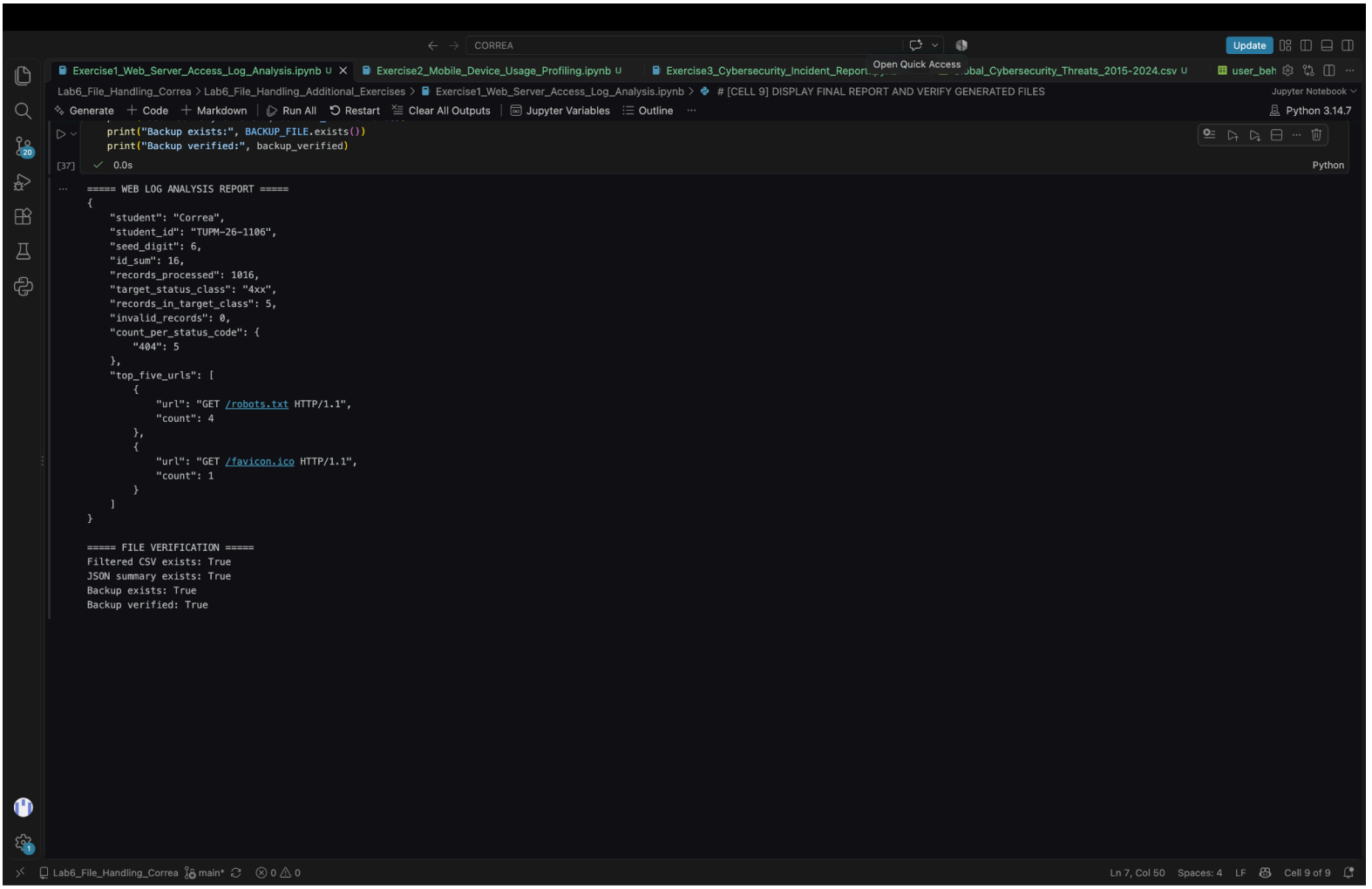
Lab6_File_Handling_Correa main* 0 0 Ln 4, Col 35 Spaces: 4 Spaces: 4 LF Cell 8 of 9


CORREA
Update
Exercise1_Web_Server_Access_Log_Analysis.ipynb U
Exercise2_Mobile_Device_Usage_Profiling.ipynb U
Exercise3_Cybersecurity_Incident_Report.ipynb U
Global_Cybersecurity_Threats_2015-2024.csv U
user_beh
Lab6_File_Handling_Correa > Lab6_File_Handling_Additional_Exercises > Exercise1_Web_Server_Access_Log_Analysis.ipynb > # [CELL 8] CREATE AND VERIFY JSON BACKUP
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.7
[CELL 5] READ CSV LINE BY LINE AND FILTER HTTP STATUS CODES
status_counts = Counter()
url_counts = Counter()
records_processed = 0
records_in_target_class = 0
invalid_records = 0
with open(INPUT_FILE, "r", encoding="utf-8-sig", newline="") as file:
reader = csv.DictReader(file)
columns = {
str(name).strip().lower(): name
for name in reader.fieldnames
}
ip_col = columns.get("ip")
time_col = columns.get("time")
url_col = columns.get("url")
status_col = columns.get("staus") or columns.get("status")
if not all([ip_col, time_col, url_col, status_col]):
raise ValueError("Required columns are missing.")
with open(FILTERED_FILE, "w", encoding="utf-8", newline="") as output:
fieldnames = ["IP", "Time", "URL", "Staus"]
writer = csv.DictWriter(output, fieldnames=fieldnames)
writer.writeheader()
for row in reader:
if records_processed >= RECORDS_TO_PROCESS:
break
records_processed += 1
ip = (row.get(ip_col) or "").strip()
timestamp = (row.get(time_col) or "").strip()
url = (row.get(url_col) or "").strip()
status = (row.get(status_col) or "").strip()
if not all([ip, timestamp, url, status]):
invalid_records += 1
continue
match = re.search(r"\b([1-5]\d{2})\b", status)
[33] 0.0s
Python
Lab6_File_Handling_Correa main*
Ln 4, Col 35
Spaces: 4
Spaces: 4
LF
Cell 8 of 9

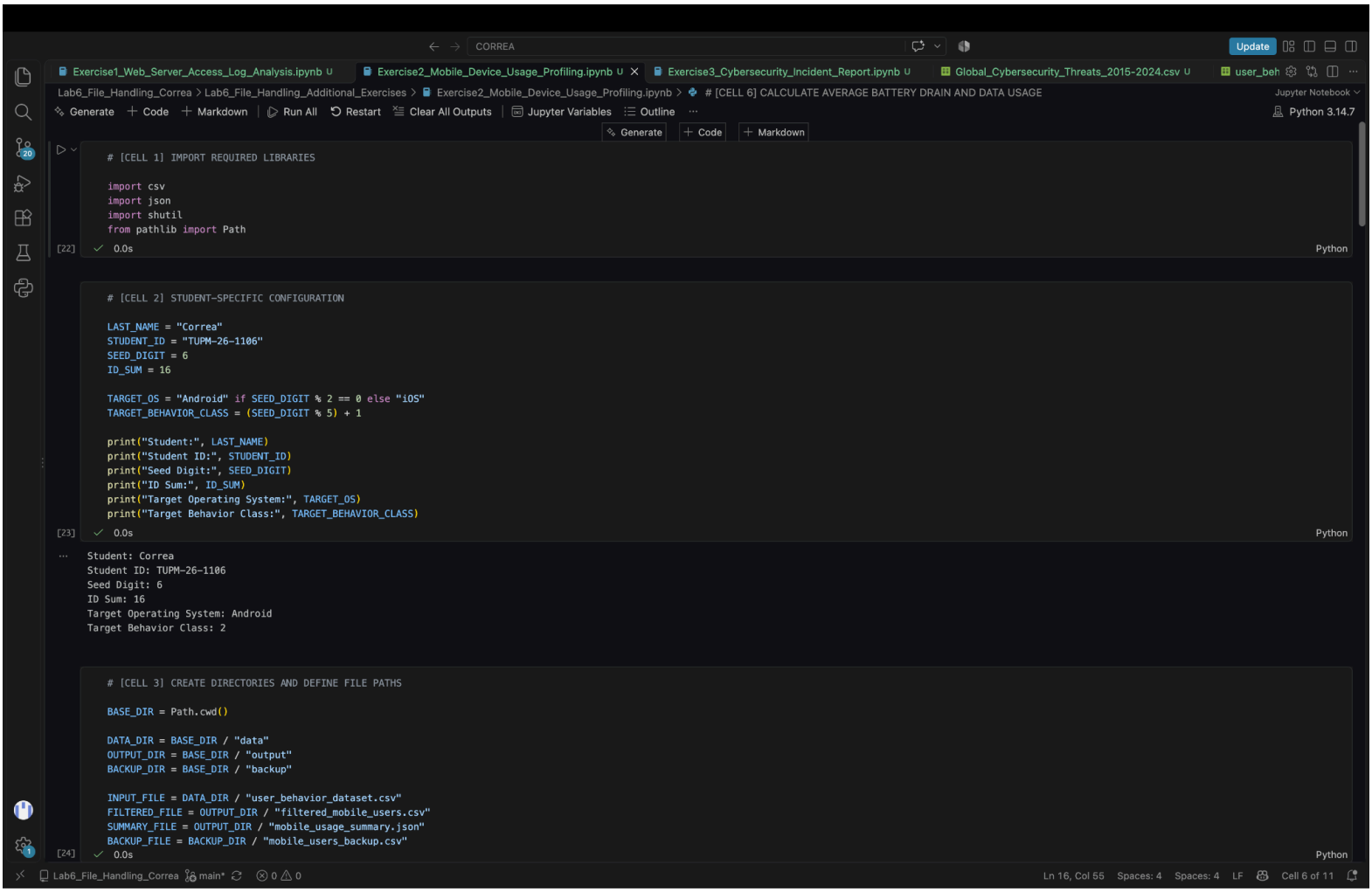

CORREA
Update
Exercise1_Web_Server_Access_Log_Analysis.ipynb U
Exercise2_Mobile_Device_Usage_Profiling.ipynb U
Exercise3_Cybersecurity_Incident_Report.ipynb U
Global_Cybersecurity_Threats_2015-2024.csv U
user_beh
Lab6_File_Handling_Correa > Lab6_File_Handling_Additional_Exercises > Exercise1_Web_Server_Access_Log_Analysis.ipynb > # [CELL 8] CREATE AND VERIFY JSON BACKUP
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.7
break
records_processed += 1
ip = (row.get(ip_col) or "").strip()
timestamp = (row.get(time_col) or "").strip()
url = (row.get(url_col) or "").strip()
status = (row.get(status_col) or "").strip()
if not all([ip, timestamp, url, status]):
invalid_records += 1
continue
match = re.search(r"\b([1-5]\d{2})\b", status)
if not match:
invalid_records += 1
continue
status_code = int(match.group(1))
status_class = status_code // 100
if status_class == TARGET_STATUS_CLASS:
records_in_target_class += 1
status_counts[status_code] += 1
url_counts[url] += 1
writer.writerow({
"IP": ip,
"Time": timestamp,
"URL": url,
"Staus": status_code
})
print("Log processing completed!")
print("Records processed:", records_processed)
print("Records in target class:", records_in_target_class)
print("Invalid records:", invalid_records)
[33] 0.0s
Python
Log processing completed!
Records processed: 1016
Records in target class: 5
Invalid records: 0
[CELL 6] COUNT STATUS CODES AND IDENTIFY TOP FIVE URLS
top_five_urls = url_counts.most_common(5)
[34] 0.0s
Python


CORREA
Update
Exercise1_Web_Server_Access_Log_Analysis.ipynb U
Exercise2_Mobile_Device_Usage_Profiling.ipynb U
Exercise3_Cybersecurity_Incident_Report.ipynb U
Global_Cybersecurity_Threats_2015-2024.csv U
user_beh
Lab6_File_Handling_Correa > Lab6_File_Handling_Additional_Exercises > Exercise1_Web_Server_Access_Log_Analysis.ipynb > # [CELL 8] CREATE AND VERIFY JSON BACKUP
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.7
[CELL 6] COUNT STATUS CODES AND IDENTIFY TOP FIVE URLS
top_five_urls = url_counts.most_common(5)
print("Count per status code:")
for code, count in sorted(status_counts.items()):
print(code, ":", count)
print("\nFive most requested URLs:")
for url, count in top_five_urls:
print(url, ":", count)
[34]
0.0s
Python
Count per status code:
404 : 5
Five most requested URLs:
GET /robots.txt HTTP/1.1 : 4
GET /favicon.ico HTTP/1.1 : 1
[CELL 7] CREATE JSON PROCESSING SUMMARY
summary = {
"student": LAST_NAME,
"student_id": STUDENT_ID,
"seed_digit": SEED_DIGIT,
"id_sum": ID_SUM,
"records_processed": records_processed,
"target_status_class": f"{TARGET_STATUS_CLASS}xx",
"records_in_target_class": records_in_target_class,
"invalid_records": invalid_records,
"count_per_status_code": dict(status_counts),
"top_five_urls": [
{"url": url, "count": count}
for url, count in top_five_urls
]
}
with open(SUMMARY_FILE, "w", encoding="utf-8") as file:
json.dump(summary, file, indent=4)
print("JSON summary created successfully!")
[35]
0.0s
Python
JSON summary created successfully!
Lab6_File_Handling_Correa
main*
Ln 7, Col 50
Spaces: 4
Spaces: 4
LF
Cell 8 of 9


CORREA
Update
Exercise1_Web_Server_Access_Log_Analysis.ipynb U
Exercise2_Mobile_Device_Usage_Profiling.ipynb U
Exercise3_Cybersecurity_Incident_Report.ipynb U
Global_Cybersecurity_Threats_2015-2024.csv U
user_beh
Lab6_File_Handling_Correa > Lab6_File_Handling_Additional_Exercises > Exercise1_Web_Server_Access_Log_Analysis.ipynb > # [CELL 8] CREATE AND VERIFY JSON BACKUP
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.7
[CELL 8] CREATE AND VERIFY JSON BACKUP
shutil.copy2(SUMMARY_FILE, BACKUP_FILE)
backup_verified = (
BACKUP_FILE.exists()
and SUMMARY_FILE.read_bytes() == BACKUP_FILE.read_bytes()
)
print("Backup exists:", BACKUP_FILE.exists())
print("Backup verified:", backup_verified)
[36]
0.0s
Python
Backup exists: True
Backup verified: True
[CELL 9] DISPLAY FINAL REPORT AND VERIFY GENERATED FILES
with open(SUMMARY_FILE, "r", encoding="utf-8") as file:
final_summary = json.load(file)
print("===== WEB LOG ANALYSIS REPORT =====")
print(json.dumps(final_summary, indent=4))
print("\n===== FILE VERIFICATION =====")
print("Filtered CSV exists:", FILTERED_FILE.exists())
print("JSON summary exists:", SUMMARY_FILE.exists())
print("Backup exists:", BACKUP_FILE.exists())
print("Backup verified:", backup_verified)
[37]
0.0s
Python
===== WEB LOG ANALYSIS REPORT =====
{
"student": "Correa",
"student_id": "TUPM-26-1106",
"seed_digit": 6,
"id_sum": 16,
"records_processed": 1016,
"target_status_class": "4xx",
"records_in_target_class": 5,
"invalid_records": 0,
"count_per_status_code": {
"404": 5
},
"top_five_urls": [
{
"url": "GET /robots.txt HTTP/1.1",
"count": 4
},


CORREA
Update
Open Quick Access
Exercise1_Web_Server_Access_Log_Analysis.ipynb U
Exercise2_Mobile_Device_Usage_Profiling.ipynb U
Exercise3_Cybersecurity_Incident_Report...
bal_Cybersecurity_Threats_2015-2024.csv U
user_beh
Lab6_File_Handling_Correa > Lab6_File_Handling_Additional_Exercises > Exercise1_Web_Server_Access_Log_Analysis.ipynb > # [CELL 9] DISPLAY FINAL REPORT AND VERIFY GENERATED FILES
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.7
print("Backup exists:", BACKUP_FILE.exists())
print("Backup verified:", backup_verified)
[37]
0.0s
Python
===== WEB LOG ANALYSIS REPORT =====
{
"student": "Correa",
"student_id": "TUPM-26-1106",
"seed_digit": 6,
"id_sum": 16,
"records_processed": 1016,
"target_status_class": "4xx",
"records_in_target_class": 5,
"invalid_records": 0,
"count_per_status_code": {
"404": 5
},
"top_five_urls": [
{
"url": "GET /robots.txt HTTP/1.1",
"count": 4
},
{
"url": "GET /favicon.ico HTTP/1.1",
"count": 1
}
]
}
===== FILE VERIFICATION =====
Filtered CSV exists: True
JSON summary exists: True
Backup exists: True
Backup verified: True
Lab6_File_Handling_Correa
main*
0 0
Ln 7, Col 50
Spaces: 4
LF
Cell 9 of 9

EXERCISE #2 MOBILE DEVICE USAGE PROFILING



```
# [CELL 1] IMPORT REQUIRED LIBRARIES

import csv
import json
import shutil
from pathlib import Path
```

[22] ✓ 0.0s Python

```
# [CELL 2] STUDENT-SPECIFIC CONFIGURATION

LAST_NAME = "Correa"
STUDENT_ID = "TUPM-26-1106"
SEED_DIGIT = 6
ID_SUM = 16

TARGET_OS = "Android" if SEED_DIGIT % 2 == 0 else "iOS"
TARGET_BEHAVIOR_CLASS = (SEED_DIGIT % 5) + 1

print("Student:", LAST_NAME)
print("Student ID:", STUDENT_ID)
print("Seed Digit:", SEED_DIGIT)
print("ID Sum:", ID_SUM)
print("Target Operating System:", TARGET_OS)
print("Target Behavior Class:", TARGET_BEHAVIOR_CLASS)
```

[23] ✓ 0.0s Python

```
Student: Correa
Student ID: TUPM-26-1106
Seed Digit: 6
ID Sum: 16
Target Operating System: Android
Target Behavior Class: 2
```

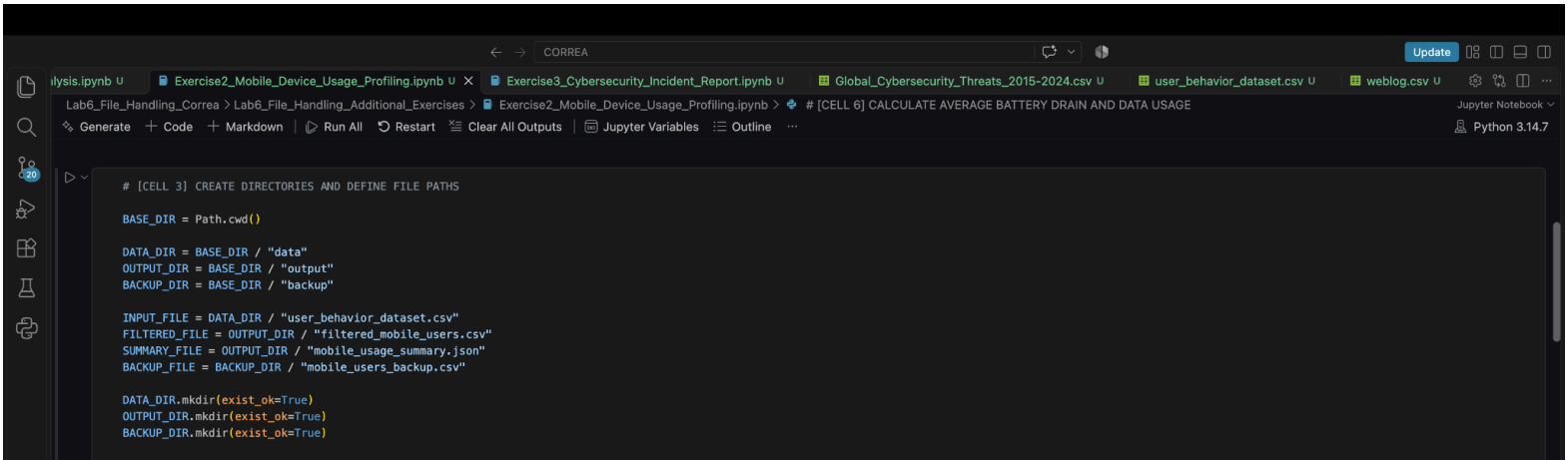
```
# [CELL 3] CREATE DIRECTORIES AND DEFINE FILE PATHS

BASE_DIR = Path.cwd()

DATA_DIR = BASE_DIR / "data"
OUTPUT_DIR = BASE_DIR / "output"
BACKUP_DIR = BASE_DIR / "backup"

INPUT_FILE = DATA_DIR / "user_behavior_dataset.csv"
FILTERED_FILE = OUTPUT_DIR / "filtered_mobile_users.csv"
SUMMARY_FILE = OUTPUT_DIR / "mobile_usage_summary.json"
BACKUP_FILE = BACKUP_DIR / "mobile_users_backup.csv"
```

[24] ✓ 0.0s Python



```
# [CELL 3] CREATE DIRECTORIES AND DEFINE FILE PATHS

BASE_DIR = Path.cwd()

DATA_DIR = BASE_DIR / "data"
OUTPUT_DIR = BASE_DIR / "output"
BACKUP_DIR = BASE_DIR / "backup"

INPUT_FILE = DATA_DIR / "user_behavior_dataset.csv"
FILTERED_FILE = OUTPUT_DIR / "filtered_mobile_users.csv"
SUMMARY_FILE = OUTPUT_DIR / "mobile_usage_summary.json"
BACKUP_FILE = BACKUP_DIR / "mobile_users_backup.csv"

DATA_DIR.mkdir(exist_ok=True)
OUTPUT_DIR.mkdir(exist_ok=True)
BACKUP_DIR.mkdir(exist_ok=True)

print("Project Directory:", BASE_DIR)
print("Input File:", INPUT_FILE)
print("Input File Exists:", INPUT_FILE.exists())
print("Output Directory:", OUTPUT_DIR)
print("Backup Directory:", BACKUP_DIR)
```

[24] ✓ 0.0s Python

```
Project Directory: /Users/jherielle/Downloads/CORREA/Lab6_File_Handling_Correa/Lab6_File_Handling_Additional_Exercises
Input File: /Users/jherielle/Downloads/CORREA/Lab6_File_Handling_Correa/Lab6_File_Handling_Additional_Exercises/data/user_behavior_dataset.csv
Input File Exists: True
Output Directory: /Users/jherielle/Downloads/CORREA/Lab6_File_Handling_Correa/Lab6_File_Handling_Additional_Exercises/output
Backup Directory: /Users/jherielle/Downloads/CORREA/Lab6_File_Handling_Correa/Lab6_File_Handling_Additional_Exercises/backup
```

```
# [CELL 4] INSPECT DATASET AND PREVIEW RECORDS

with open(INPUT_FILE, "r", encoding="utf-8-sig", newline="") as file:
    reader = csv.DictReader(file)

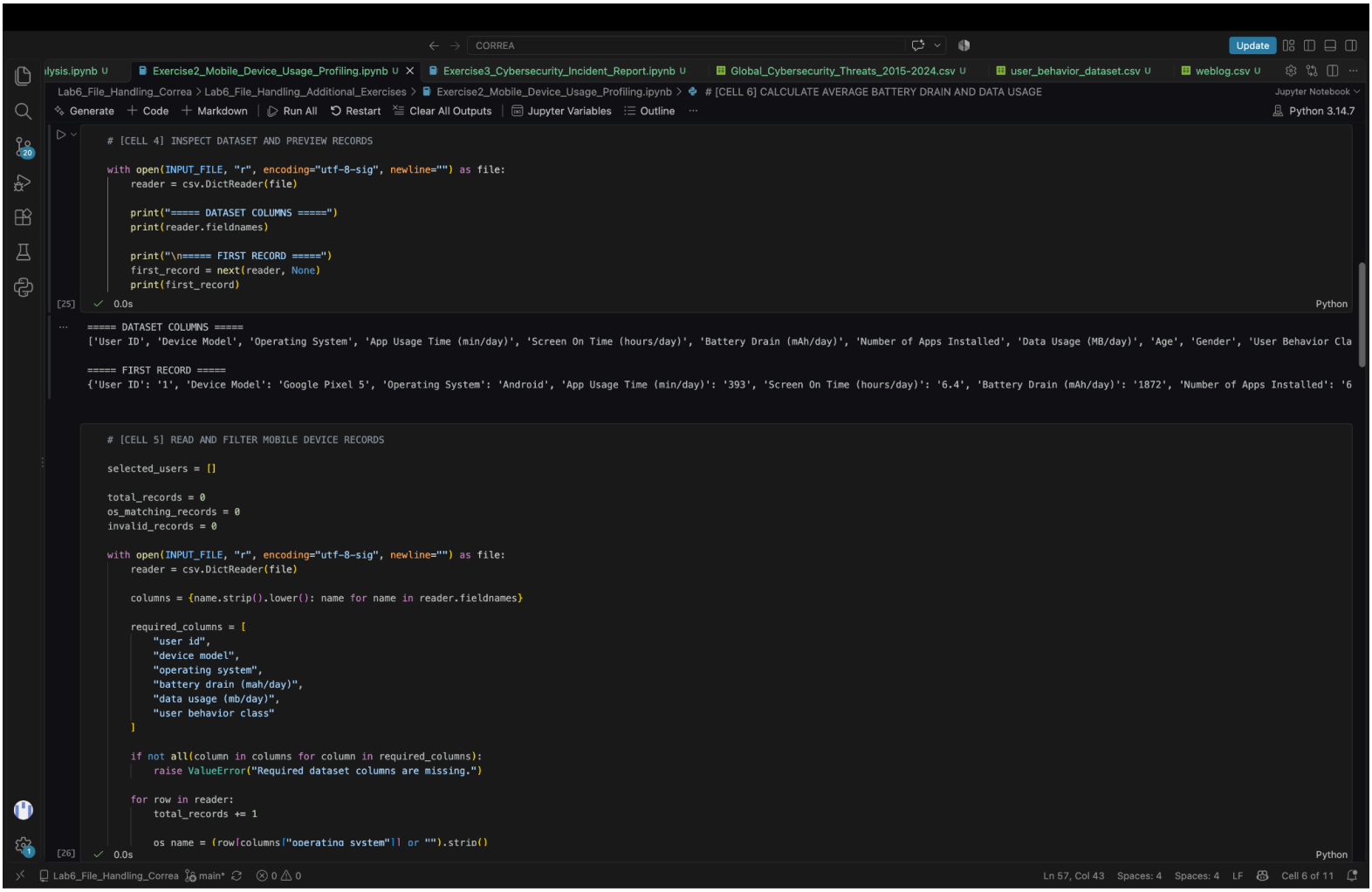
    print("===== DATASET COLUMNS =====")
    print(reader.fieldnames)

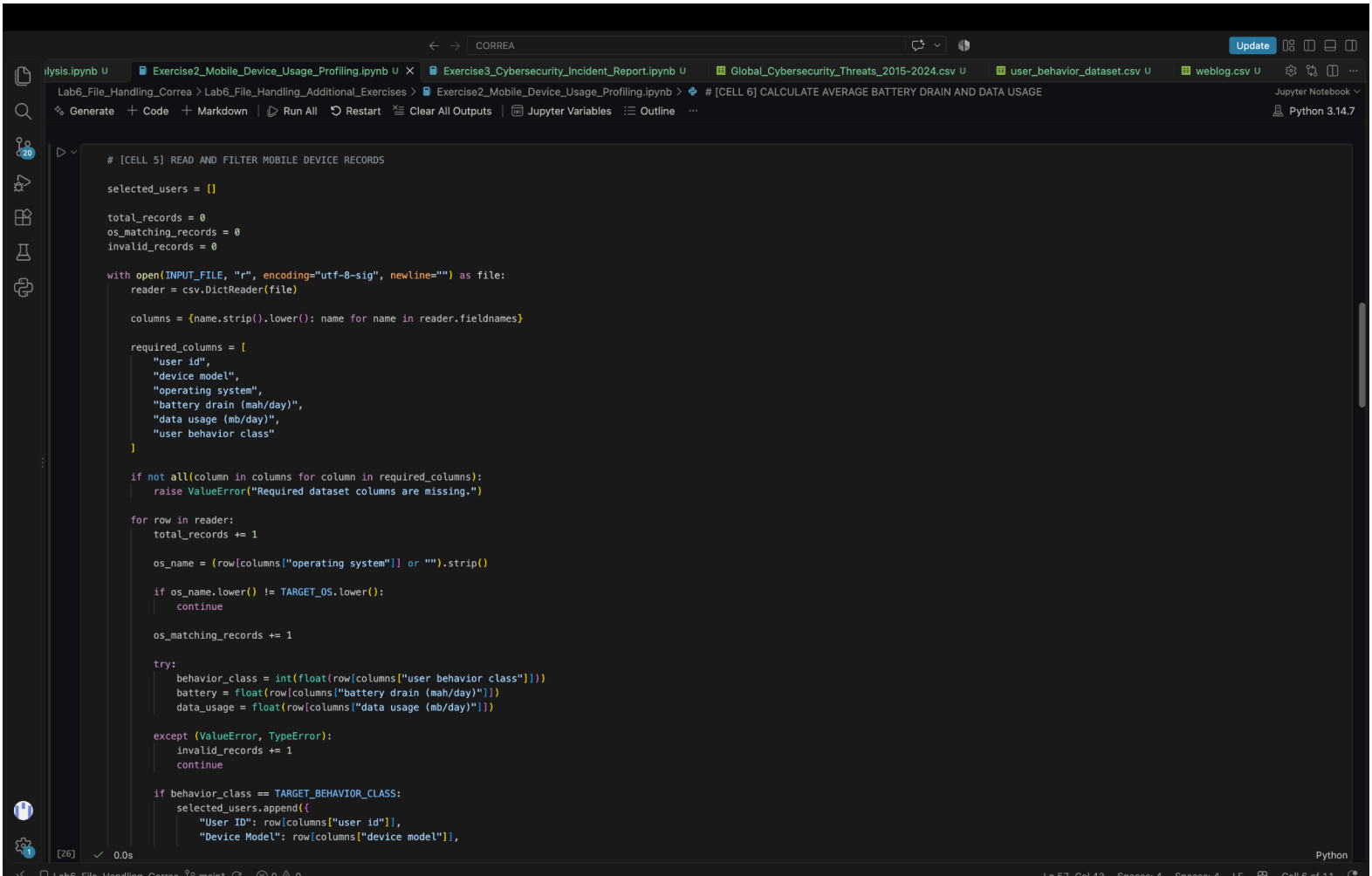
    print("\n===== FIRST RECORD =====")
    first_record = next(reader, None)
    print(first_record)
```

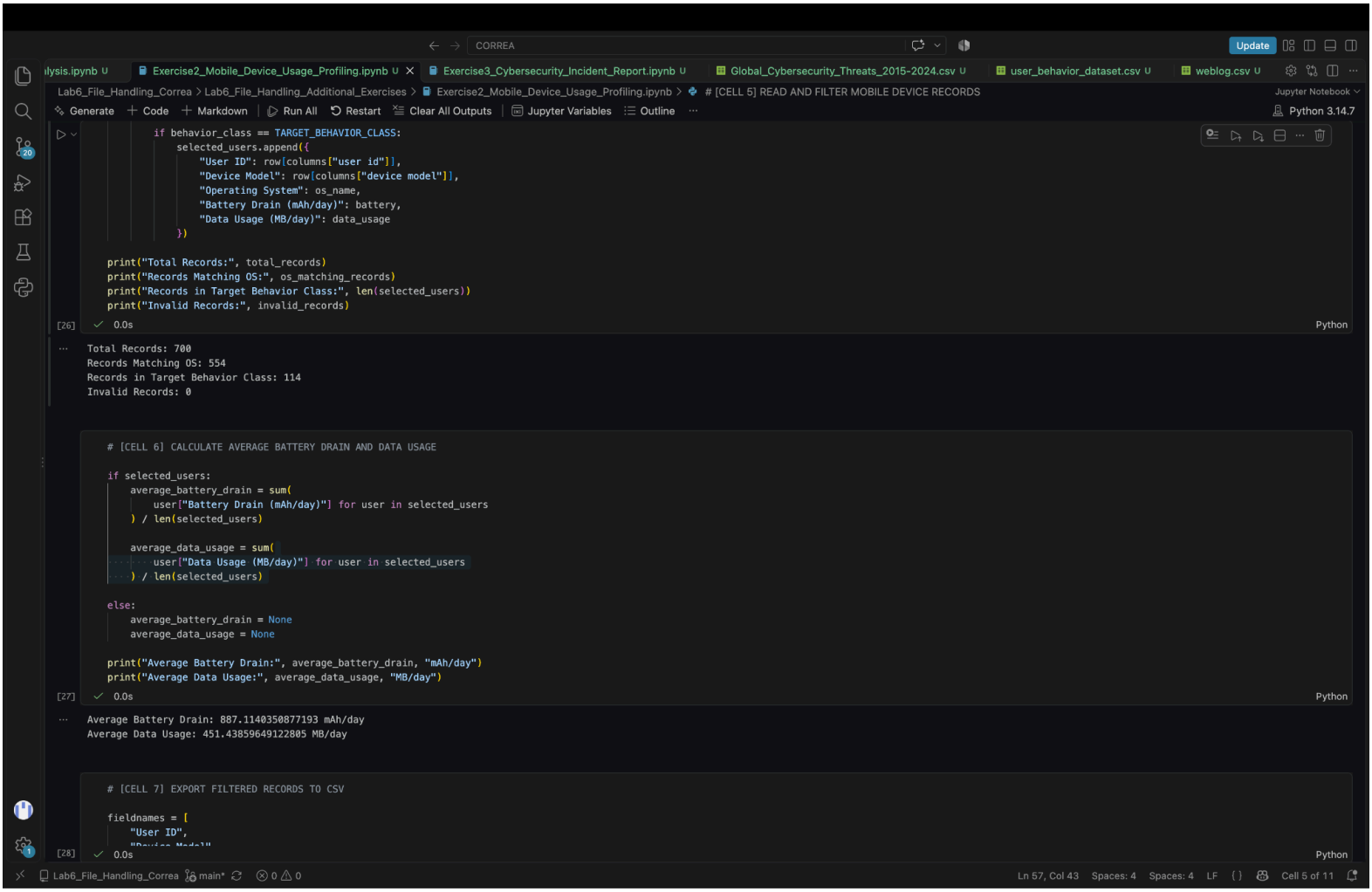
[25] ✓ 0.0s Python

```
===== DATASET COLUMNS =====
['User ID', 'Device Model', 'Operating System', 'App Usage Time (min/day)', 'Screen On Time (hours/day)', 'Battery Drain (mAh/day)', 'Number of Apps Installed', 'Data Usage (MB/day)', 'Age', 'Gender', 'User Behavior Cla

===== FIRST RECORD =====
```


CORREA
Update
Exercise2_Mobile_Device_Usage_Profiling.ipynb
Exercise3_Cybersecurity_Incident_Report.ipynb
Global_Cybersecurity_Threats_2015-2024.csv
user_behavior_dataset.csv
weblog.csv
Lab6_File_Handling_Correa > Lab6_File_Handling_Additional_Exercises > Exercise2_Mobile_Device_Usage_Profiling.ipynb > # [CELL 6] CALCULATE AVERAGE BATTERY DRAIN AND DATA USAGE
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.7
[CELL 4] INSPECT DATASET AND PREVIEW RECORDS
with open(INPUT_FILE, "r", encoding="utf-8-sig", newline="") as file:
reader = csv.DictReader(file)
print("===== DATASET COLUMNS =====")
print(reader.fieldnames)
print("\n===== FIRST RECORD =====")
first_record = next(reader, None)
print(first_record)
[25]
0.0s
Python
===== DATASET COLUMNS =====
['User ID', 'Device Model', 'Operating System', 'App Usage Time (min/day)', 'Screen On Time (hours/day)', 'Battery Drain (mAh/day)', 'Number of Apps Installed', 'Data Usage (MB/day)', 'Age', 'Gender', 'User Behavior Cla
===== FIRST RECORD =====
{'User ID': '1', 'Device Model': 'Google Pixel 5', 'Operating System': 'Android', 'App Usage Time (min/day)': '393', 'Screen On Time (hours/day)': '6.4', 'Battery Drain (mAh/day)': '1872', 'Number of Apps Installed': '6
[CELL 5] READ AND FILTER MOBILE DEVICE RECORDS
selected_users = []
total_records = 0
os_matching_records = 0
invalid_records = 0
with open(INPUT_FILE, "r", encoding="utf-8-sig", newline="") as file:
reader = csv.DictReader(file)
columns = {name.strip().lower(): name for name in reader.fieldnames}
required_columns = [
"user id",
"device model",
"operating system",
"battery drain (mah/day)",
"data usage (mb/day)",
"user behavior class"
]
if not all(column in columns for column in required_columns):
raise ValueError("Required dataset columns are missing.")
for row in reader:
total_records += 1
os_name = (row[columns["operating system"]] or "").strip()
[26]
0.0s
Python
Lab6_File_Handling_Correa
main*
Ln 57, Col 43
Spaces: 4
Spaces: 4
LF
Cell 6 of 11


CORREA
Update
Exercise2_Mobile_Device_Usage_Profiling.ipynb
Exercise3_Cybersecurity_Incident_Report.ipynb
Global_Cybersecurity_Threats_2015-2024.csv
user_behavior_dataset.csv
weblog.csv
Lab6_File_Handling_Correa > Lab6_File_Handling_Additional_Exercises > Exercise2_Mobile_Device_Usage_Profiling.ipynb > # [CELL 6] CALCULATE AVERAGE BATTERY DRAIN AND DATA USAGE
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.7
[CELL 5] READ AND FILTER MOBILE DEVICE RECORDS
selected_users = []
total_records = 0
os_matching_records = 0
invalid_records = 0
with open(INPUT_FILE, "r", encoding="utf-8-sig", newline="") as file:
reader = csv.DictReader(file)
columns = {name.strip().lower(): name for name in reader.fieldnames}
required_columns = [
"user id",
"device model",
"operating system",
"battery drain (mah/day)",
"data usage (mb/day)",
"user behavior class"
]
if not all(column in columns for column in required_columns):
raise ValueError("Required dataset columns are missing.")
for row in reader:
total_records += 1
os_name = (row[columns["operating system"]] or "").strip()
if os_name.lower() != TARGET_OS.lower():
continue
os_matching_records += 1
try:
behavior_class = int(float(row[columns["user behavior class"]]))
battery = float(row[columns["battery drain (mah/day)"]])
data_usage = float(row[columns["data usage (mb/day)"]])
except (ValueError, TypeError):
invalid_records += 1
continue
if behavior_class == TARGET_BEHAVIOR_CLASS:
selected_users.append({
"User ID": row[columns["user id"]],
"Device Model": row[columns["device model"]],
[26]
0.0s
Python



```
            if behavior_class == TARGET_BEHAVIOR_CLASS:
                selected_users.append({
                    "User ID": row[columns["user id"]],
                    "Device Model": row[columns["device model"]],
                    "Operating System": os_name,
                    "Battery Drain (mAh/day)": battery,
                    "Data Usage (MB/day)": data_usage
                })

    print("Total Records:", total_records)
    print("Records Matching OS:", os_matching_records)
    print("Records in Target Behavior Class:", len(selected_users))
    print("Invalid Records:", invalid_records)
```

```
Total Records: 700
Records Matching OS: 554
Records in Target Behavior Class: 114
Invalid Records: 0
```

```
# [CELL 6] CALCULATE AVERAGE BATTERY DRAIN AND DATA USAGE

if selected_users:
    average_battery_drain = sum(
        user["Battery Drain (mAh/day)"] for user in selected_users
    ) / len(selected_users)

    average_data_usage = sum(
        user["Data Usage (MB/day)"] for user in selected_users
    ) / len(selected_users)

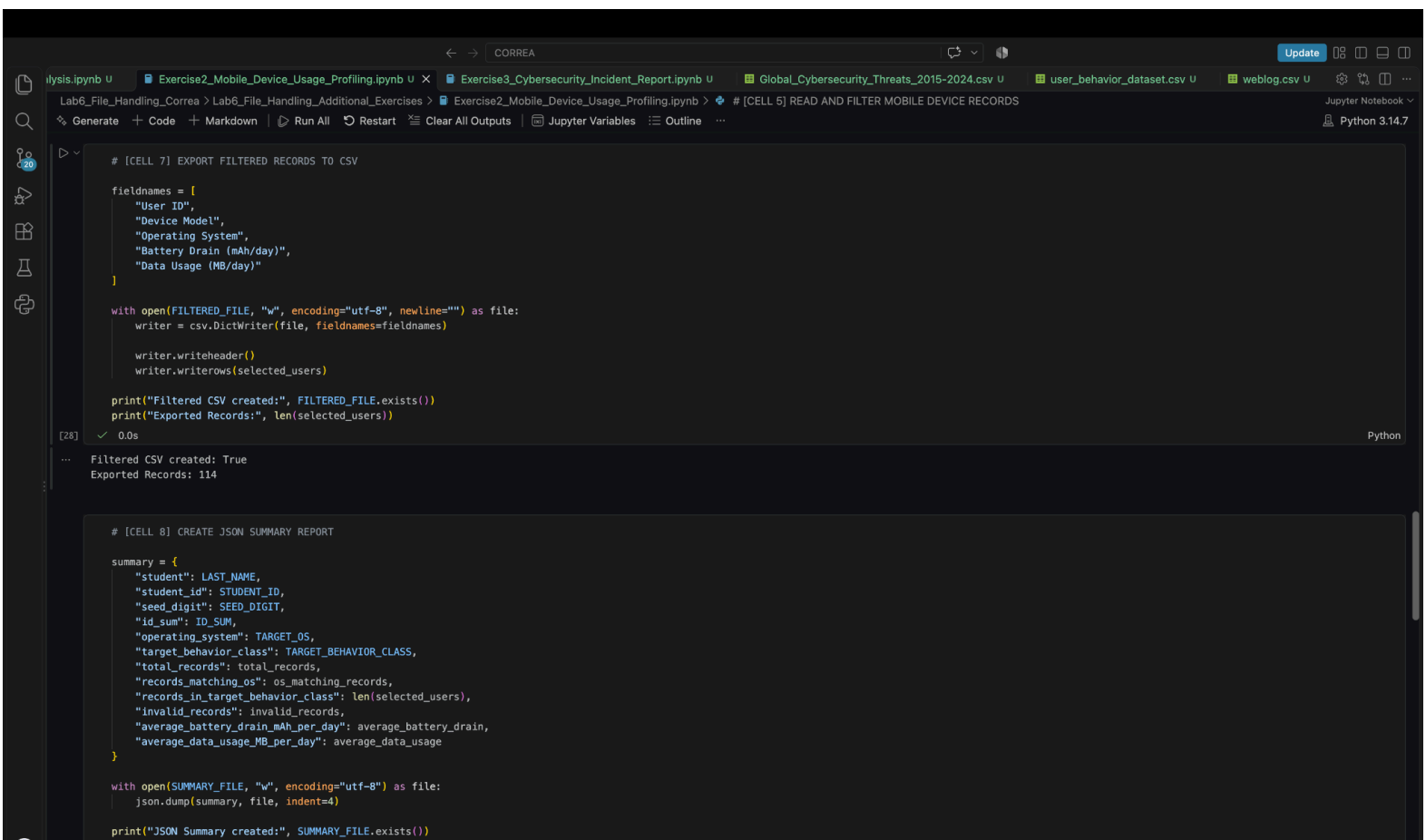
else:
    average_battery_drain = None
    average_data_usage = None

print("Average Battery Drain:", average_battery_drain, "mAh/day")
print("Average Data Usage:", average_data_usage, "MB/day")
```

```
Average Battery Drain: 887.1140350877193 mAh/day
Average Data Usage: 451.43859649122805 MB/day
```

```
# [CELL 7] EXPORT FILTERED RECORDS TO CSV

fieldnames = [
    "User ID",
```



```
# [CELL 7] EXPORT FILTERED RECORDS TO CSV

fieldnames = [
    "User ID",
    "Device Model",
    "Operating System",
    "Battery Drain (mAh/day)",
    "Data Usage (MB/day)"
]

with open(FILTERED_FILE, "w", encoding="utf-8", newline="") as file:
    writer = csv.DictWriter(file, fieldnames=fieldnames)

    writer.writeheader()
    writer.writerows(selected_users)

print("Filtered CSV created:", FILTERED_FILE.exists())
print("Exported Records:", len(selected_users))
```

```
Filtered CSV created: True
Exported Records: 114
```

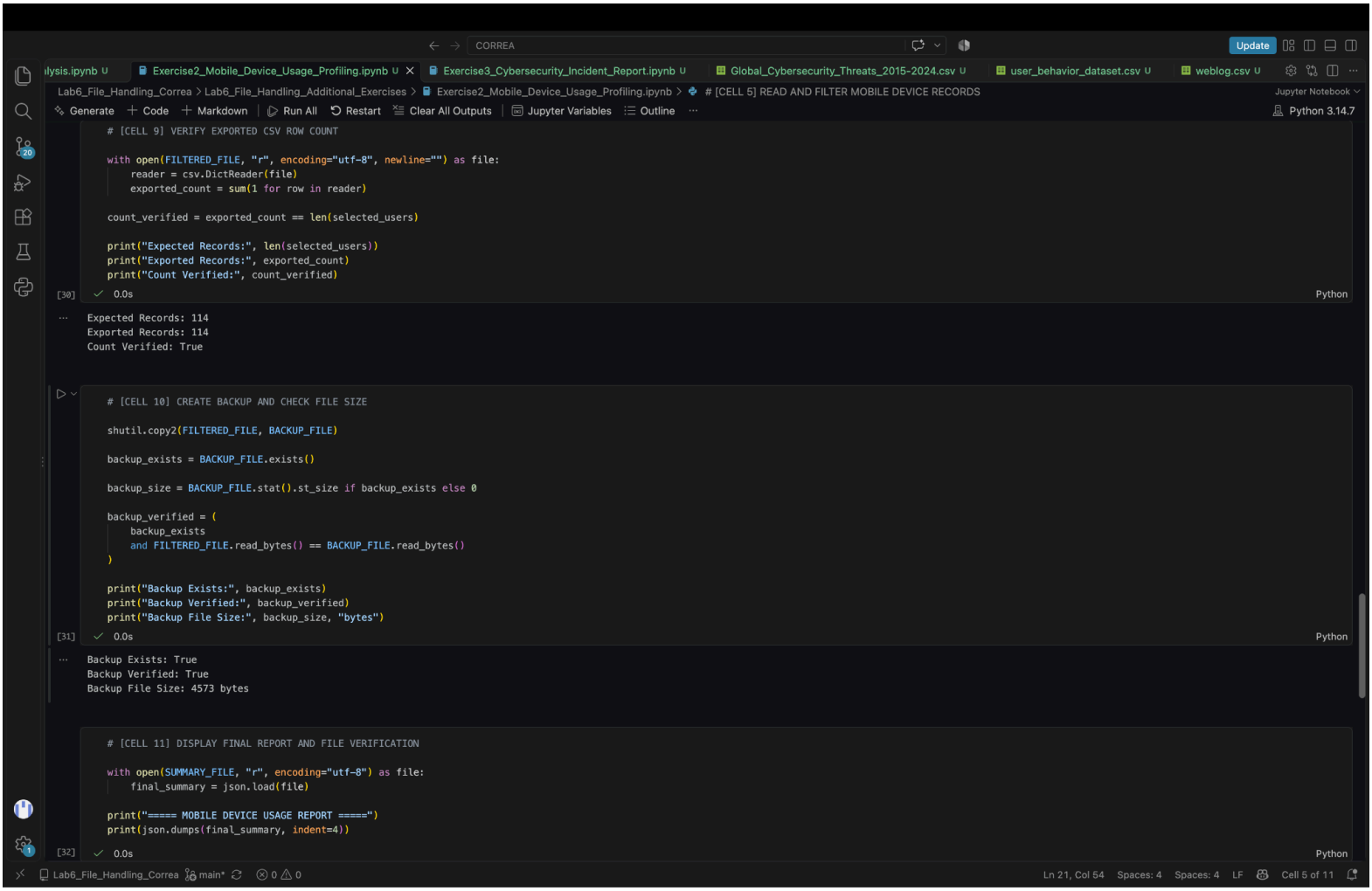
```
# [CELL 8] CREATE JSON SUMMARY REPORT

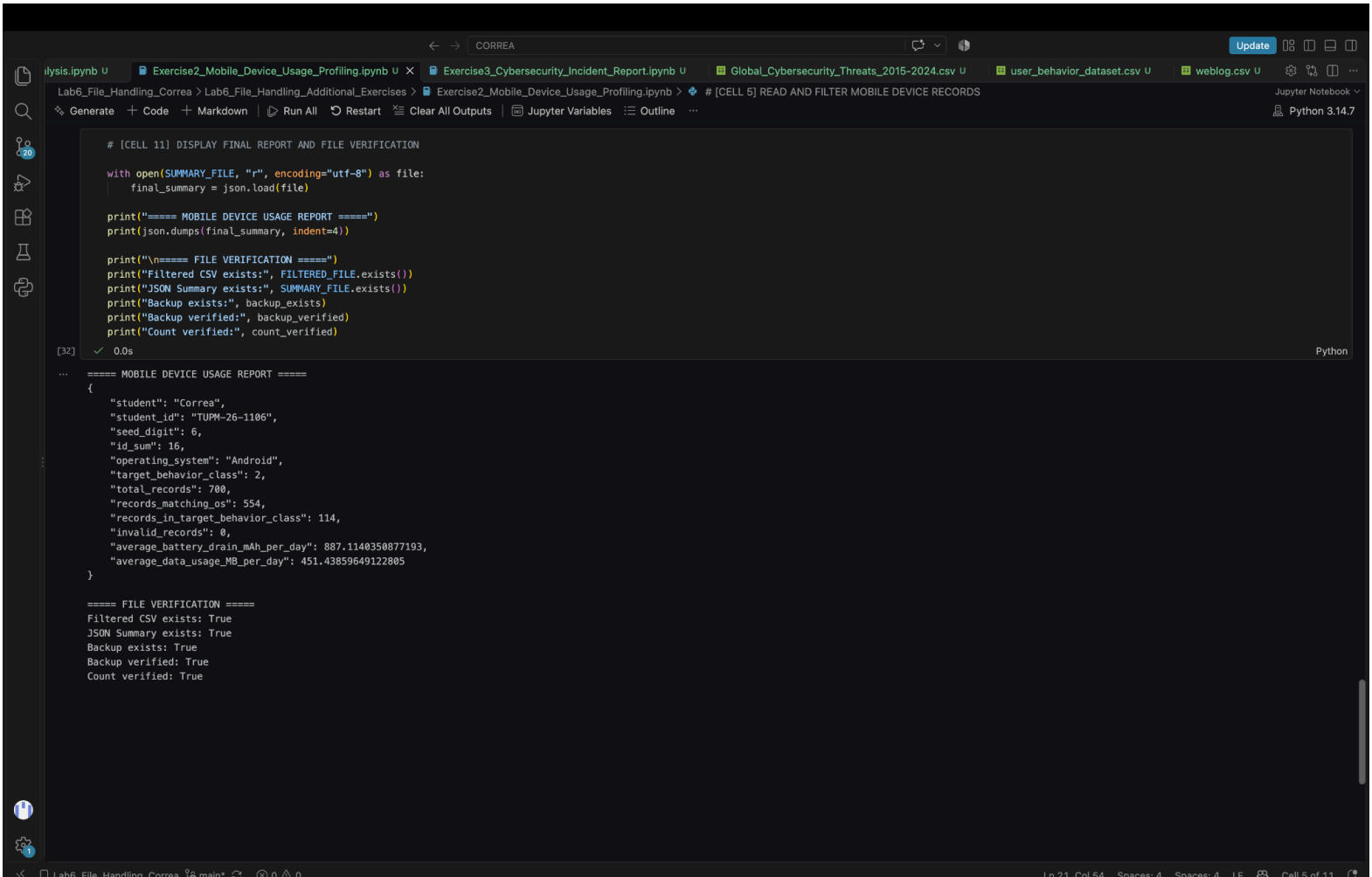
summary = {
    "student": LAST_NAME,
    "student_id": STUDENT_ID,
    "seed_digit": SEED_DIGIT,
    "id_sum": ID_SUM,
    "operating_system": TARGET_OS,
    "target_behavior_class": TARGET_BEHAVIOR_CLASS,
    "total_records": total_records,
    "records_matching_os": os_matching_records,
    "records_in_target_behavior_class": len(selected_users),
    "invalid_records": invalid_records,
    "average_battery_drain_mAh_per_day": average_battery_drain,
    "average_data_usage_MB_per_day": average_data_usage
}

with open(SUMMARY_FILE, "w", encoding="utf-8") as file:
    json.dump(summary, file, indent=4)

print("JSON Summary created:", SUMMARY_FILE.exists())
```

```
JSON Summary created: True
```


CORREA
Update
Exercise2_Mobile_Device_Usage_Profiling.ipynb
Exercise3_Cybersecurity_Incident_Report.ipynb
Global_Cybersecurity_Threats_2015-2024.csv
user_behavior_dataset.csv
weblog.csv
Lab6_File_Handling_Correa > Lab6_File_Handling_Additional_Exercises > Exercise2_Mobile_Device_Usage_Profiling.ipynb > # [CELL 5] READ AND FILTER MOBILE DEVICE RECORDS
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.7
[CELL 9] VERIFY EXPORTED CSV ROW COUNT
with open(FILTERED_FILE, "r", encoding="utf-8", newline="") as file:
reader = csv.DictReader(file)
exported_count = sum(1 for row in reader)
count_verified = exported_count == len(selected_users)
print("Expected Records:", len(selected_users))
print("Exported Records:", exported_count)
print("Count Verified:", count_verified)
[30]
0.0s
Python
Expected Records: 114
Exported Records: 114
Count Verified: True
[CELL 10] CREATE BACKUP AND CHECK FILE SIZE
shutil.copy2(FILTERED_FILE, BACKUP_FILE)
backup_exists = BACKUP_FILE.exists()
backup_size = BACKUP_FILE.stat().st_size if backup_exists else 0
backup_verified = (
backup_exists
and FILTERED_FILE.read_bytes() == BACKUP_FILE.read_bytes()
)
print("Backup Exists:", backup_exists)
print("Backup Verified:", backup_verified)
print("Backup File Size:", backup_size, "bytes")
[31]
0.0s
Python
Backup Exists: True
Backup Verified: True
Backup File Size: 4573 bytes
[CELL 11] DISPLAY FINAL REPORT AND FILE VERIFICATION
with open(SUMMARY_FILE, "r", encoding="utf-8") as file:
final_summary = json.load(file)
print("===== MOBILE DEVICE USAGE REPORT =====")
print(json.dumps(final_summary, indent=4))
[32]
0.0s
Python
Lab6_File_Handling_Correa
main*
Ln 21, Col 54
Spaces: 4
LF
Cell 5 of 11


CORREA
Update
Exercise2_Mobile_Device_Usage_Profiling.ipynb
Exercise3_Cybersecurity_Incident_Report.ipynb
Global_Cybersecurity_Threats_2015-2024.csv
user_behavior_dataset.csv
weblog.csv
Lab6_File_Handling_Correa > Lab6_File_Handling_Additional_Exercises > Exercise2_Mobile_Device_Usage_Profiling.ipynb > # [CELL 5] READ AND FILTER MOBILE DEVICE RECORDS
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.7
[CELL 11] DISPLAY FINAL REPORT AND FILE VERIFICATION
with open(SUMMARY_FILE, "r", encoding="utf-8") as file:
final_summary = json.load(file)
print("===== MOBILE DEVICE USAGE REPORT =====")
print(json.dumps(final_summary, indent=4))
print("\n===== FILE VERIFICATION =====")
print("Filtered CSV exists:", FILTERED_FILE.exists())
print("JSON Summary exists:", SUMMARY_FILE.exists())
print("Backup exists:", backup_exists)
print("Backup verified:", backup_verified)
print("Count verified:", count_verified)
[32]
0.0s
Python
===== MOBILE DEVICE USAGE REPORT =====
{
"student": "Correa",
"student_id": "TUPM-26-1106",
"seed_digit": 6,
"id_sum": 16,
"operating_system": "Android",
"target_behavior_class": 2,
"total_records": 700,
"records_matching_os": 554,
"records_in_target_behavior_class": 114,
"invalid_records": 0,
"average_battery_drain_mAh_per_day": 887.1140350877193,
"average_data_usage_MB_per_day": 451.43859649122805
}
===== FILE VERIFICATION =====
Filtered CSV exists: True
JSON Summary exists: True
Backup exists: True
Backup verified: True
Count verified: True

EXERCISE # 3 CYBERSECURITY INCIDENT REPORT

```python
# [CELL 1] IMPORT REQUIRED LIBRARIES

import csv
import json
import shutil
from pathlib import Path
from collections import Counter
```

[26] ✓ 0.0s Python

```python
# [CELL 2] STUDENT-SPECIFIC CONFIGURATION

LAST_NAME = "Correa"
STUDENT_ID = "TUPM-26-1106"
SEED_DIGIT = 6
ID_SUM = 16

TARGET_YEAR = 2015 + SEED_DIGIT

print("Student:", LAST_NAME)
print("Student ID:", STUDENT_ID)
print("Seed Digit:", SEED_DIGIT)
print("ID Sum:", ID_SUM)
print("Target Year:", TARGET_YEAR)
```

[27] ✓ 0.0s Python

```
Student: Correa
Student ID: TUPM-26-1106
Seed Digit: 6
ID Sum: 16
Target Year: 2021
```

```python
# [CELL 3] CREATE DIRECTORIES AND DEFINE FILE PATHS

BASE_DIR = Path.cwd()

DATA_DIR = BASE_DIR / "data"
OUTPUT_DIR = BASE_DIR / "output"
BACKUP_DIR = BASE_DIR / "backup"

INPUT_FILE = DATA_DIR / "Global_Cybersecurity_Threats_2015-2024.csv"

FILTERED_FILE = OUTPUT_DIR / "filtered_cybersecurity_incidents.csv"
TOP5_FILE = OUTPUT_DIR / "top_costliest_incidents.json"
SUMMARY_FILE = OUTPUT_DIR / "incident_summary.json"
BACKUP_FILE = BACKUP_DIR / "top_costliest_incidents_backup.json"
```

[28] ✓ 0.0s Python

```python
# [CELL 3] CREATE DIRECTORIES AND DEFINE FILE PATHS

BASE_DIR = Path.cwd()

DATA_DIR = BASE_DIR / "data"
OUTPUT_DIR = BASE_DIR / "output"
BACKUP_DIR = BASE_DIR / "backup"

INPUT_FILE = DATA_DIR / "Global_Cybersecurity_Threats_2015-2024.csv"

FILTERED_FILE = OUTPUT_DIR / "filtered_cybersecurity_incidents.csv"
TOP5_FILE = OUTPUT_DIR / "top_costliest_incidents.json"
SUMMARY_FILE = OUTPUT_DIR / "incident_summary.json"
BACKUP_FILE = BACKUP_DIR / "top_costliest_incidents_backup.json"
REPORT_FILE = OUTPUT_DIR / "cybersecurity_incident_report.txt"

DATA_DIR.mkdir(exist_ok=True)
OUTPUT_DIR.mkdir(exist_ok=True)
BACKUP_DIR.mkdir(exist_ok=True)

print("Project Directory:", BASE_DIR)
print("Input File:", INPUT_FILE)
print("Input File Exists:", INPUT_FILE.exists())
print("Output Directory:", OUTPUT_DIR)
print("Backup Directory:", BACKUP_DIR)
```

[28] ✓ 0.0s Python

```
Project Directory: /Users/jherielle/Downloads/CORREA/Lab6_File_Handling_Correa/Lab6_File_Handling_Additional_Exercises
Input File: /Users/jherielle/Downloads/CORREA/Lab6_File_Handling_Correa/Lab6_File_Handling_Additional_Exercises/data/Global_Cybersecurity_Threats_2015-2024.csv
Input File Exists: True
Output Directory: /Users/jherielle/Downloads/CORREA/Lab6_File_Handling_Correa/Lab6_File_Handling_Additional_Exercises/output
Backup Directory: /Users/jherielle/Downloads/CORREA/Lab6_File_Handling_Correa/Lab6_File_Handling_Additional_Exercises/backup
```

```python
# [CELL 4] INSPECT DATASET AND PREVIEW RECORDS

with open(INPUT_FILE, "r", encoding="utf-8-sig", newline="") as file:
    reader = csv.DictReader(file)

    print("===== DATASET COLUMNS =====")
    print(reader.fieldnames)

    print("\n===== FIRST RECORD =====")
    first_record = next(reader, None)
    print(first_record)
```

[29] ✓ 0.0s Python

```
===== DATASET COLUMNS =====
['Country', 'Year', 'Attack Type', 'Target Industry', 'Financial Loss (in Million $)', 'Number of Affected Users', 'Attack Source', 'Security Vulnerability Type', 'Defense Mechanism Used', 'Incident Resolution Time (in
```

```python
# [CELL 4] INSPECT DATASET AND PREVIEW RECORDS

with open(INPUT_FILE, "r", encoding="utf-8-sig", newline="") as file:
    reader = csv.DictReader(file)

    print("===== DATASET COLUMNS =====")
    print(reader.fieldnames)

    print("\n===== FIRST RECORD =====")
    first_record = next(reader, None)
    print(first_record)
```

[29] ✓ 0.0s Python

```
===== DATASET COLUMNS =====
['Country', 'Year', 'Attack Type', 'Target Industry', 'Financial Loss (in Million $)', 'Number of Affected Users', 'Attack Source', 'Security Vulnerability Type', 'Defense Mechanism Used', 'Incident Resolution Time (in

===== FIRST RECORD =====
{'Country': 'China', 'Year': '2019', 'Attack Type': 'Phishing', 'Target Industry': 'Education', 'Financial Loss (in Million $)': '80.53', 'Number of Affected Users': '773169', 'Attack Source': 'Hacker Group', 'Security
```

```python
# [CELL 5] READ AND FILTER CYBERSECURITY INCIDENTS

selected_incidents = []

total_records = 0
invalid_year_records = 0
invalid_financial_records = 0

with open(INPUT_FILE, "r", encoding="utf-8-sig", newline="") as file:
    reader = csv.DictReader(file)

    for row in reader:
        total_records += 1

        try:
            year = int(row["Year"])
        except (ValueError, TypeError):
            invalid_year_records += 1
            continue

        if year != TARGET_YEAR:
            continue

        try:
            financial_loss = float(row["Financial Loss (in Million $)"])

            if financial_loss < 0:
                raise ValueError
```

[30] ✓ 0.0s Python

CORREA
Update
filing.ipynb U | Exercise3_Cybersecurity_Incident_Report.ipynb U | Global_Cybersecurity_Threats_2015-2024.csv U | user_behavior_dataset.csv U | weblog.csv U | household_power_consumption.txt .../Lab6_File_Handlir
Lab6_File_Handling_Correa > Lab6_File_Handling_Additional_Exercises > Exercise3_Cybersecurity_Incident_Report.ipynb > # [CELL 14] VERIFY ALL OUTPUT FILES
Jupyter Notebook
Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline
Python 3.14.7

```python
            try:
                financial_loss = float(row["Financial Loss (in Million $)"])

                if financial_loss < 0:
                    raise ValueError

            except (ValueError, TypeError):
                invalid_financial_records += 1
                continue

            incident = {
                "Country": row["Country"],
                "Year": year,
                "Attack Type": row["Attack Type"],
                "Target Industry": row["Target Industry"],
                "Financial Loss (in Million $)": financial_loss,
                "Number of Affected Users": row["Number of Affected Users"],
                "Attack Source": row["Attack Source"],
                "Security Vulnerability Type": row["Security Vulnerability Type"],
                "Defense Mechanism Used": row["Defense Mechanism Used"],
                "Incident Resolution Time (in Hours)": row["Incident Resolution Time (in Hours)"]
            }

            selected_incidents.append(incident)

    print("Total Records:", total_records)
    print("Target Year:", TARGET_YEAR)
    print("Incidents in Target Year:", len(selected_incidents))
    print("Invalid Year Records:", invalid_year_records)
    print("Invalid Financial Records:", invalid_financial_records)
```

[30] ✓ 0.0s Python

```
Total Records: 3000
Target Year: 2021
Incidents in Target Year: 299
Invalid Year Records: 0
Invalid Financial Records: 0
```

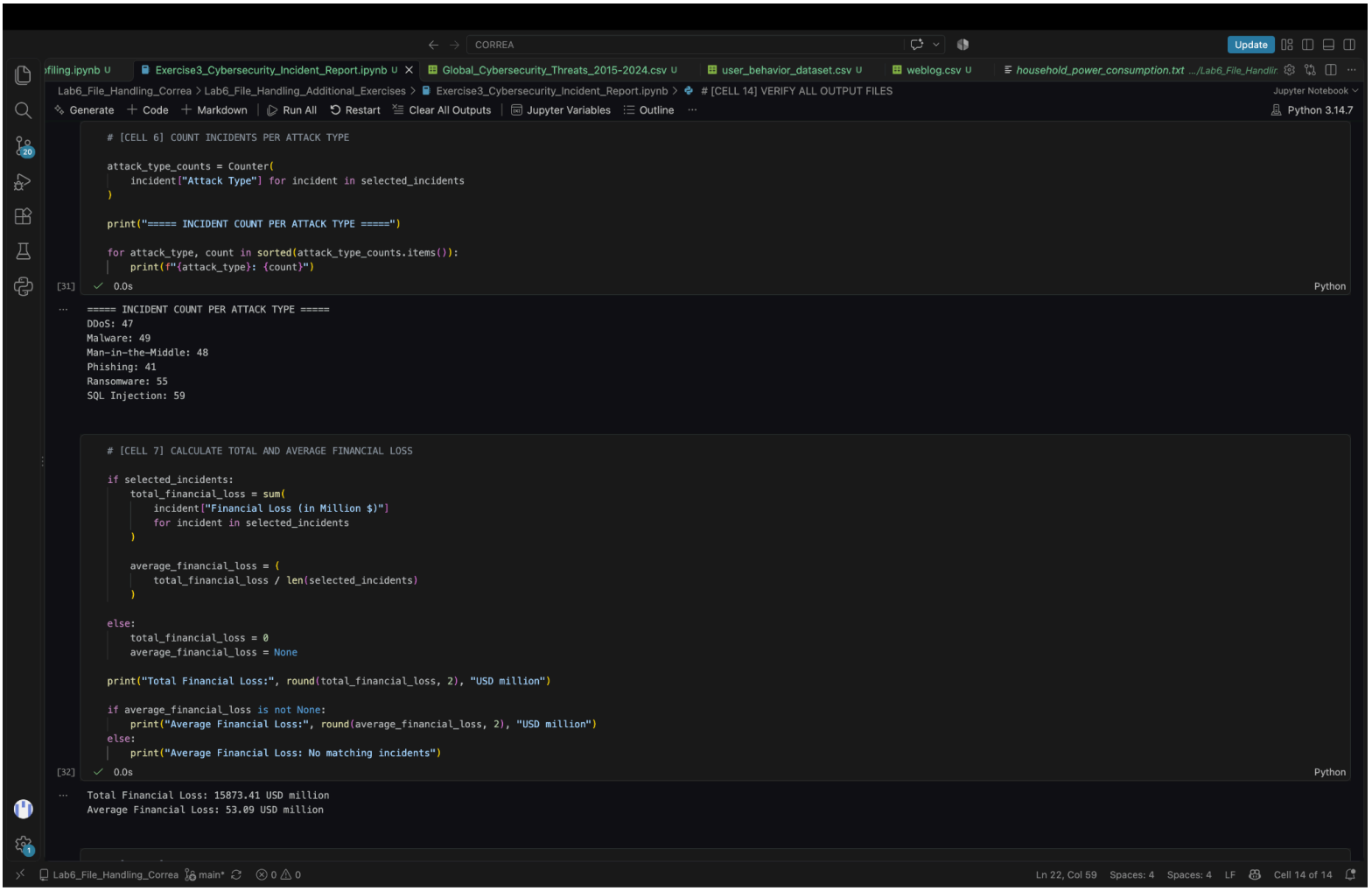
```python
# [CELL 6] COUNT INCIDENTS PER ATTACK TYPE

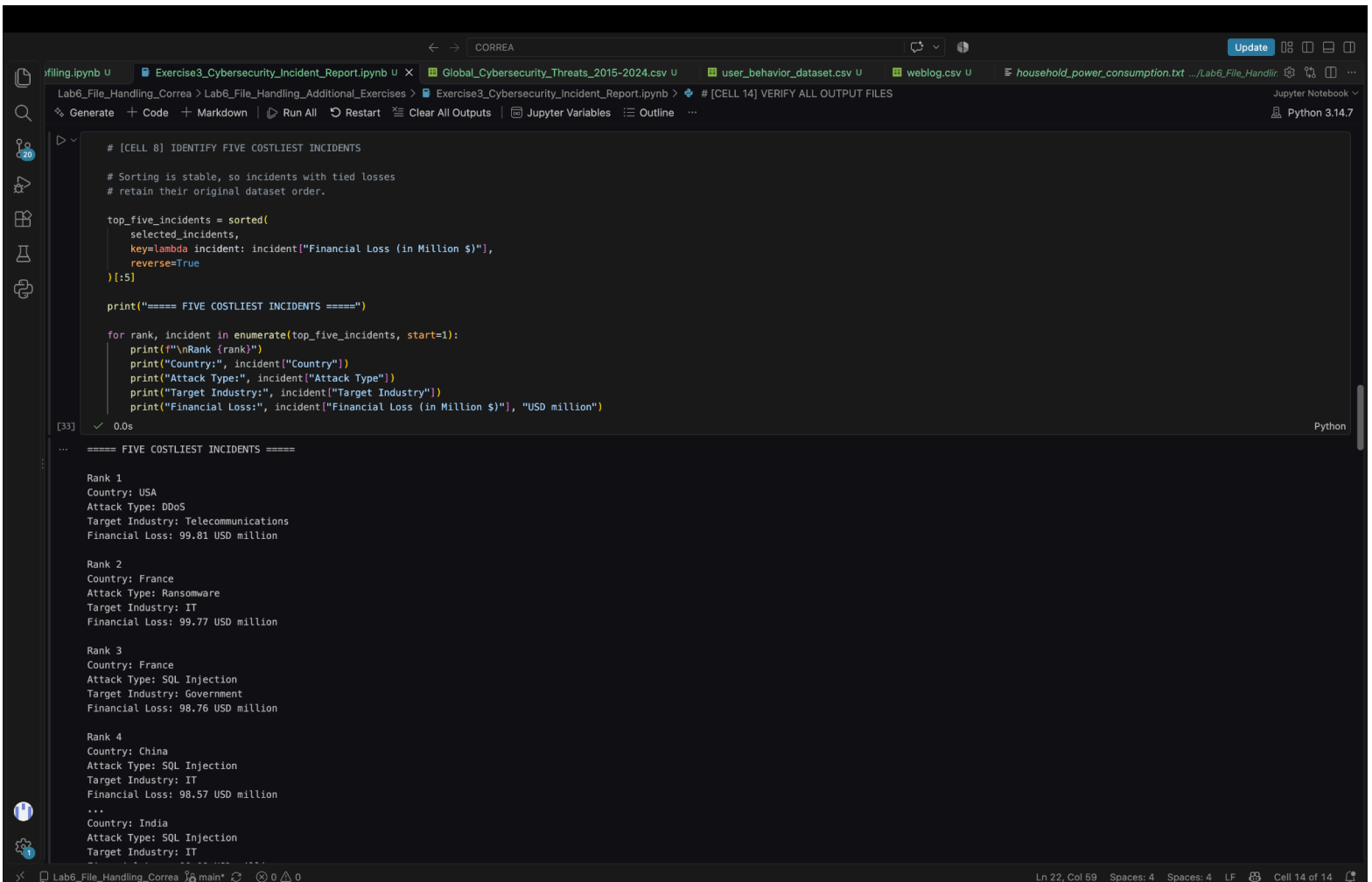
attack_type_counts = Counter(
    incident["Attack Type"] for incident in selected_incidents
)

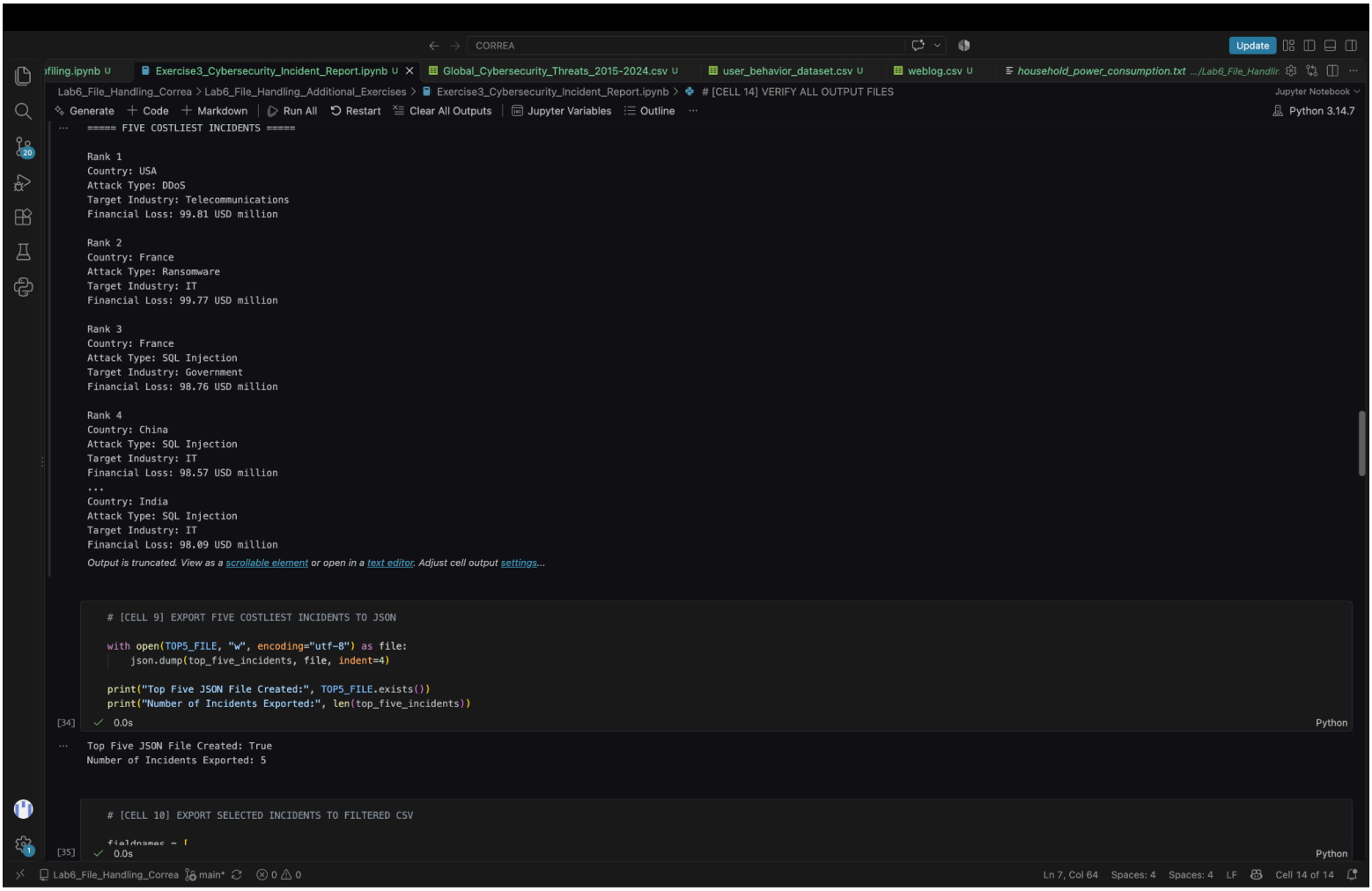
print("===== INCIDENT COUNT PER ATTACK TYPE =====")

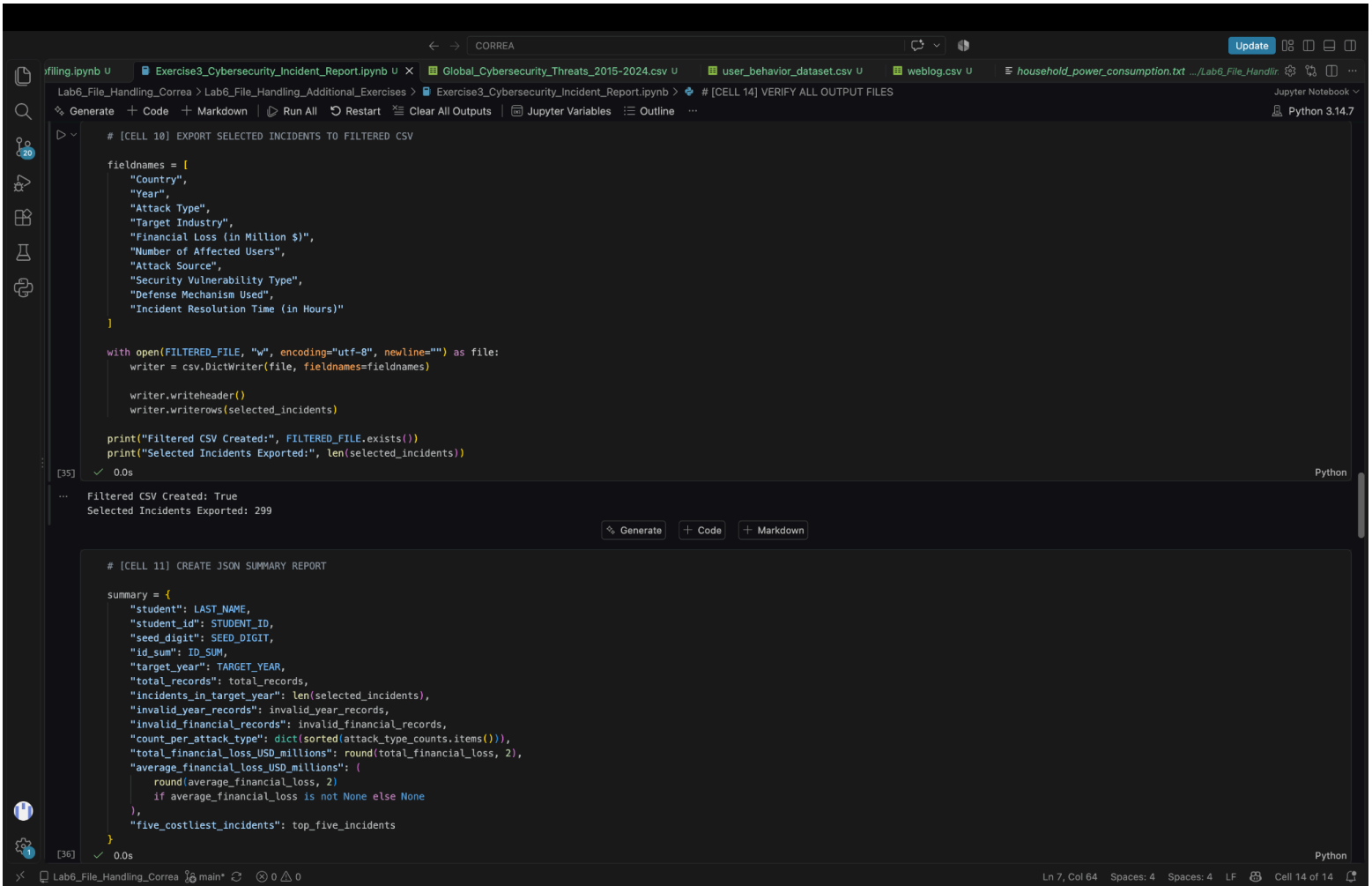
for attack_type, count in sorted(attack_type_counts.items()):
```

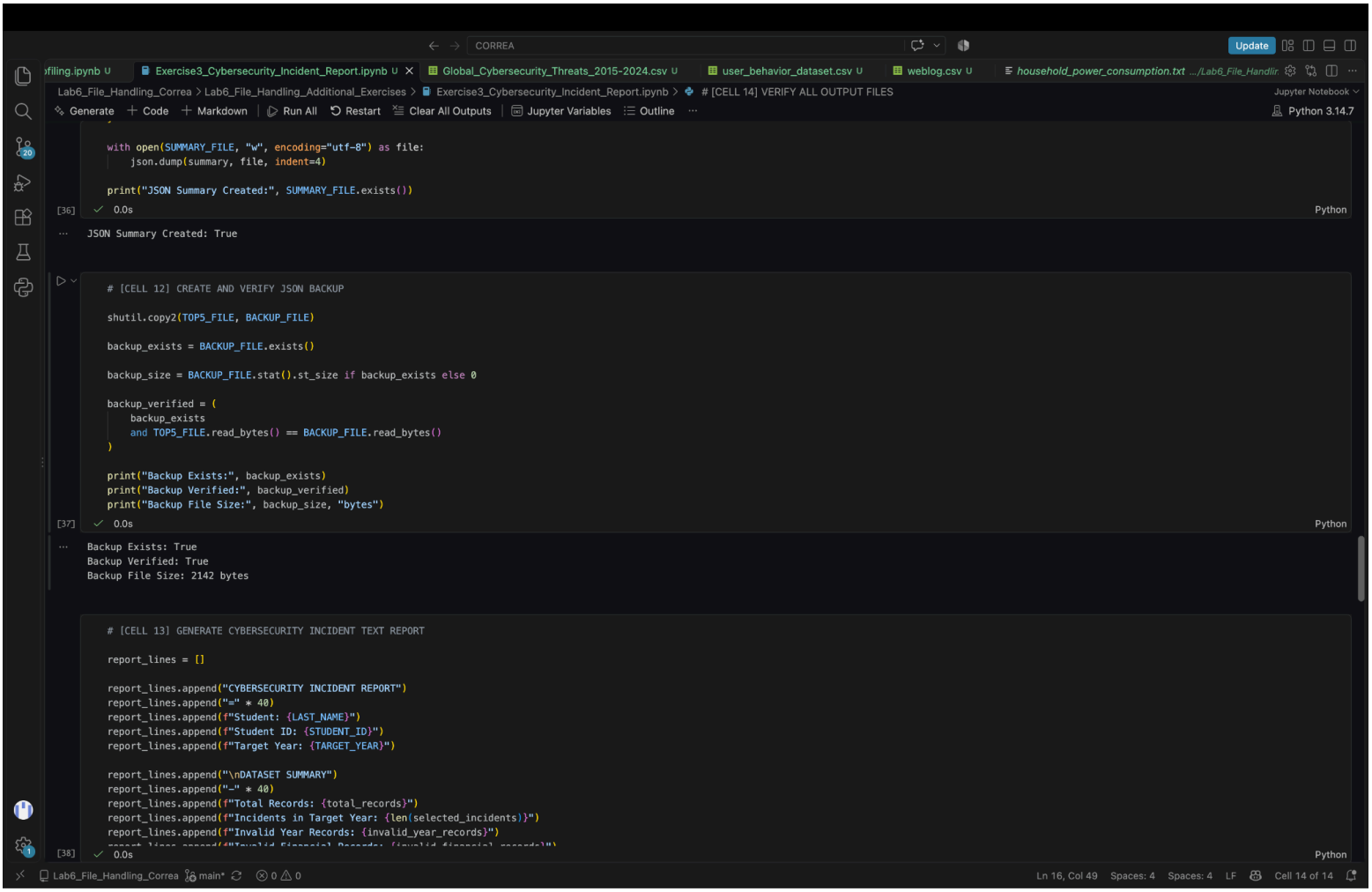
[31] ✓ 0.0s Python


CORREA
Update
filing.ipynb U
Exercise3_Cybersecurity_Incident_Report.ipynb U
Global_Cybersecurity_Threats_2015-2024.csv U
user_behavior_dataset.csv U
weblog.csv U
household_power_consumption.txt .../Lab6_File_Handlir
Lab6_File_Handling_Correa > Lab6_File_Handling_Additional_Exercises > Exercise3_Cybersecurity_Incident_Report.ipynb > # [CELL 14] VERIFY ALL OUTPUT FILES
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.7
[CELL 6] COUNT INCIDENTS PER ATTACK TYPE
attack_type_counts = Counter(
incident["Attack Type"] for incident in selected_incidents
)
print("===== INCIDENT COUNT PER ATTACK TYPE =====")
for attack_type, count in sorted(attack_type_counts.items()):
print(f"{attack_type}: {count}")
[31]
0.0s
Python
===== INCIDENT COUNT PER ATTACK TYPE =====
DDoS: 47
Malware: 49
Man-in-the-Middle: 48
Phishing: 41
Ransomware: 55
SQL Injection: 59
[CELL 7] CALCULATE TOTAL AND AVERAGE FINANCIAL LOSS
if selected_incidents:
total_financial_loss = sum(
incident["Financial Loss (in Million $)"]
for incident in selected_incidents
)
average_financial_loss = (
total_financial_loss / len(selected_incidents)
)
else:
total_financial_loss = 0
average_financial_loss = None
print("Total Financial Loss:", round(total_financial_loss, 2), "USD million")
if average_financial_loss is not None:
print("Average Financial Loss:", round(average_financial_loss, 2), "USD million")
else:
print("Average Financial Loss: No matching incidents")
[32]
0.0s
Python
Total Financial Loss: 15873.41 USD million
Average Financial Loss: 53.09 USD million
Lab6_File_Handling_Correa
main*
0 0
Ln 22, Col 59
Spaces: 4
Spaces: 4
LF
Cell 14 of 14


CORREA
Update
filing.ipynb U
Exercise3_Cybersecurity_Incident_Report.ipynb U
Global_Cybersecurity_Threats_2015-2024.csv U
user_behavior_dataset.csv U
weblog.csv U
household_power_consumption.txt .../Lab6_File_Handlir
Lab6_File_Handling_Correa > Lab6_File_Handling_Additional_Exercises > Exercise3_Cybersecurity_Incident_Report.ipynb > # [CELL 14] VERIFY ALL OUTPUT FILES
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.7
[CELL 8] IDENTIFY FIVE COSTLIEST INCIDENTS
Sorting is stable, so incidents with tied losses
retain their original dataset order.
top_five_incidents = sorted(
selected_incidents,
key=lambda incident: incident["Financial Loss (in Million $)"],
reverse=True
)[:5]
print("===== FIVE COSTLIEST INCIDENTS =====")
for rank, incident in enumerate(top_five_incidents, start=1):
print(f"\nRank {rank}")
print("Country:", incident["Country"])
print("Attack Type:", incident["Attack Type"])
print("Target Industry:", incident["Target Industry"])
print("Financial Loss:", incident["Financial Loss (in Million $)"], "USD million")
[33]
0.0s
Python
===== FIVE COSTLIEST INCIDENTS =====
Rank 1
Country: USA
Attack Type: DDoS
Target Industry: Telecommunications
Financial Loss: 99.81 USD million
Rank 2
Country: France
Attack Type: Ransomware
Target Industry: IT
Financial Loss: 99.77 USD million
Rank 3
Country: France
Attack Type: SQL Injection
Target Industry: Government
Financial Loss: 98.76 USD million
Rank 4
Country: China
Attack Type: SQL Injection
Target Industry: IT
Financial Loss: 98.57 USD million
...
Country: India
Attack Type: SQL Injection
Target Industry: IT
Lab6_File_Handling_Correa
main*
0 0
Ln 22, Col 59
Spaces: 4
Spaces: 4
LF
Cell 14 of 14


CORREA
Update
filing.ipynb U
Exercise3_Cybersecurity_Incident_Report.ipynb U
Global_Cybersecurity_Threats_2015-2024.csv U
user_behavior_dataset.csv U
weblog.csv U
household_power_consumption.txt .../Lab6_File_Handlir
Lab6_File_Handling_Correa > Lab6_File_Handling_Additional_Exercises > Exercise3_Cybersecurity_Incident_Report.ipynb > # [CELL 14] VERIFY ALL OUTPUT FILES
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.7
===== FIVE COSTLIEST INCIDENTS =====
Rank 1
Country: USA
Attack Type: DDoS
Target Industry: Telecommunications
Financial Loss: 99.81 USD million
Rank 2
Country: France
Attack Type: Ransomware
Target Industry: IT
Financial Loss: 99.77 USD million
Rank 3
Country: France
Attack Type: SQL Injection
Target Industry: Government
Financial Loss: 98.76 USD million
Rank 4
Country: China
Attack Type: SQL Injection
Target Industry: IT
Financial Loss: 98.57 USD million
...
Country: India
Attack Type: SQL Injection
Target Industry: IT
Financial Loss: 98.09 USD million
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
[CELL 9] EXPORT FIVE COSTLIEST INCIDENTS TO JSON
with open(TOP5_FILE, "w", encoding="utf-8") as file:
json.dump(top_five_incidents, file, indent=4)
print("Top Five JSON File Created:", TOP5_FILE.exists())
print("Number of Incidents Exported:", len(top_five_incidents))
[34]
0.0s
Python
Top Five JSON File Created: True
Number of Incidents Exported: 5
[CELL 10] EXPORT SELECTED INCIDENTS TO FILTERED CSV
[35]
0.0s
Python
Lab6_File_Handling_Correa
main*
0 0
Ln 7, Col 64
Spaces: 4
Spaces: 4
LF
Cell 14 of 14


CORREA
Update
filing.ipynb U
Exercise3_Cybersecurity_Incident_Report.ipynb U
Global_Cybersecurity_Threats_2015-2024.csv U
user_behavior_dataset.csv U
weblog.csv U
household_power_consumption.txt .../Lab6_File_Handlir
Lab6_File_Handling_Correa > Lab6_File_Handling_Additional_Exercises > Exercise3_Cybersecurity_Incident_Report.ipynb > # [CELL 14] VERIFY ALL OUTPUT FILES
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.7
[CELL 10] EXPORT SELECTED INCIDENTS TO FILTERED CSV
fieldnames = [
"Country",
"Year",
"Attack Type",
"Target Industry",
"Financial Loss (in Million $)",
"Number of Affected Users",
"Attack Source",
"Security Vulnerability Type",
"Defense Mechanism Used",
"Incident Resolution Time (in Hours)"
]
with open(FILTERED_FILE, "w", encoding="utf-8", newline="") as file:
writer = csv.DictWriter(file, fieldnames=fieldnames)
writer.writeheader()
writer.writerows(selected_incidents)
print("Filtered CSV Created:", FILTERED_FILE.exists())
print("Selected Incidents Exported:", len(selected_incidents))
[35]
0.0s
Python
Filtered CSV Created: True
Selected Incidents Exported: 299
Generate
Code
Markdown
[CELL 11] CREATE JSON SUMMARY REPORT
summary = {
"student": LAST_NAME,
"student_id": STUDENT_ID,
"seed_digit": SEED_DIGIT,
"id_sum": ID_SUM,
"target_year": TARGET_YEAR,
"total_records": total_records,
"incidents_in_target_year": len(selected_incidents),
"invalid_year_records": invalid_year_records,
"invalid_financial_records": invalid_financial_records,
"count_per_attack_type": dict(sorted(attack_type_counts.items())),
"total_financial_loss_USD_millions": round(total_financial_loss, 2),
"average_financial_loss_USD_millions": (
round(average_financial_loss, 2)
if average_financial_loss is not None else None
),
"five_costliest_incidents": top_five_incidents
}
[36]
0.0s
Python
Lab6_File_Handling_Correa
main*
0 0
Ln 7, Col 64
Spaces: 4
Spaces: 4
LF
Cell 14 of 14



```
with open(SUMMARY_FILE, "w", encoding="utf-8") as file:
    json.dump(summary, file, indent=4)

print("JSON Summary Created:", SUMMARY_FILE.exists())
```

```
JSON Summary Created: True
```

```
# [CELL 12] CREATE AND VERIFY JSON BACKUP

shutil.copy2(TOP5_FILE, BACKUP_FILE)

backup_exists = BACKUP_FILE.exists()

backup_size = BACKUP_FILE.stat().st_size if backup_exists else 0

backup_verified = (
    backup_exists
    and TOP5_FILE.read_bytes() == BACKUP_FILE.read_bytes()
)

print("Backup Exists:", backup_exists)
print("Backup Verified:", backup_verified)
print("Backup File Size:", backup_size, "bytes")
```

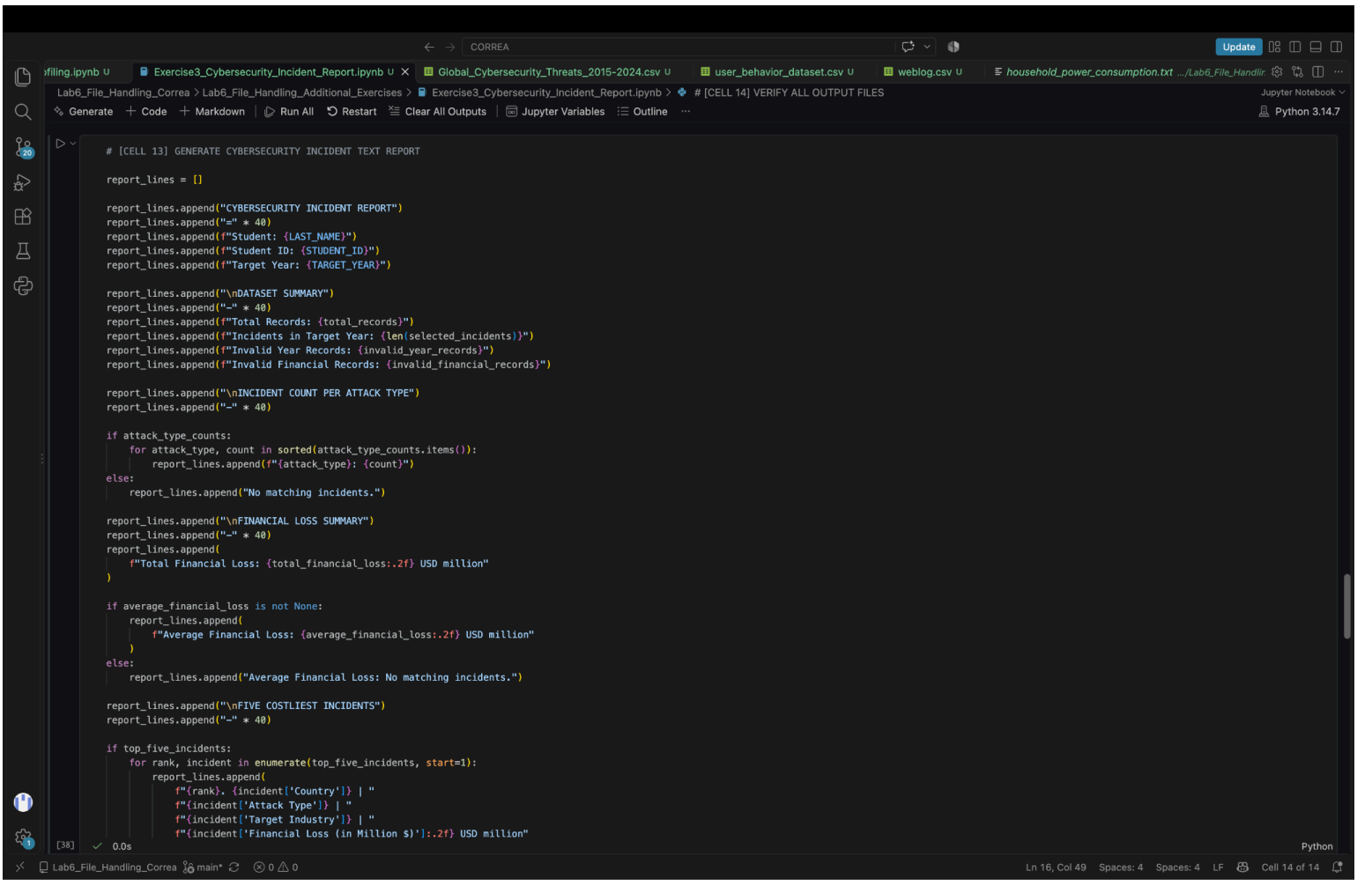
```
Backup Exists: True
Backup Verified: True
Backup File Size: 2142 bytes
```

```
# [CELL 13] GENERATE CYBERSECURITY INCIDENT TEXT REPORT

report_lines = []

report_lines.append("CYBERSECURITY INCIDENT REPORT")
report_lines.append("=" * 40)
report_lines.append(f"Student: {LAST_NAME}")
report_lines.append(f"Student ID: {STUDENT_ID}")
report_lines.append(f"Target Year: {TARGET_YEAR}")

report_lines.append("\nDATASET SUMMARY")
report_lines.append("-" * 40)
report_lines.append(f"Total Records: {total_records}")
report_lines.append(f"Incidents in Target Year: {len(selected_incidents)}")
report_lines.append(f"Invalid Year Records: {invalid_year_records}")
```



```
# [CELL 13] GENERATE CYBERSECURITY INCIDENT TEXT REPORT

report_lines = []

report_lines.append("CYBERSECURITY INCIDENT REPORT")
report_lines.append("=" * 40)
report_lines.append(f"Student: {LAST_NAME}")
report_lines.append(f"Student ID: {STUDENT_ID}")
report_lines.append(f"Target Year: {TARGET_YEAR}")

report_lines.append("\nDATASET SUMMARY")
report_lines.append("-" * 40)
report_lines.append(f"Total Records: {total_records}")
report_lines.append(f"Incidents in Target Year: {len(selected_incidents)}")
report_lines.append(f"Invalid Year Records: {invalid_year_records}")
report_lines.append(f"Invalid Financial Records: {invalid_financial_records}")

report_lines.append("\nINCIDENT COUNT PER ATTACK TYPE")
report_lines.append("-" * 40)

if attack_type_counts:
    for attack_type, count in sorted(attack_type_counts.items()):
        report_lines.append(f"{attack_type}: {count}")
else:
    report_lines.append("No matching incidents.")

report_lines.append("\nFINANCIAL LOSS SUMMARY")
report_lines.append("-" * 40)
report_lines.append(
    f"Total Financial Loss: {total_financial_loss:.2f} USD million"
)

if average_financial_loss is not None:
    report_lines.append(
        f"Average Financial Loss: {average_financial_loss:.2f} USD million"
    )
else:
    report_lines.append("Average Financial Loss: No matching incidents.")

report_lines.append("\nFIVE COSTLIEST INCIDENTS")
report_lines.append("-" * 40)

if top_five_incidents:
    for rank, incident in enumerate(top_five_incidents, start=1):
        report_lines.append(
            f"{rank}. {incident['Country']} | "
            f"{incident['Attack Type']} | "
            f"{incident['Target Industry']} | "
            f"{incident['Financial Loss (in Million $)']:.2f} USD million"
```

CORREA

Update

filing.ipynb U | Exercise3_Cybersecurity_Incident_Report.ipynb U | Global_Cybersecurity_Threats_2015-2024.csv U | user_behavior_dataset.csv U | weblog.csv U | household_power_consumption.txt .../Lab6_File_Handlir

Lab6_File_Handling_Correa > Lab6_File_Handling_Additional_Exercises > Exercise3_Cybersecurity_Incident_Report.ipynb > # [CELL 14] VERIFY ALL OUTPUT FILES

Jupyter Notebook

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ...

Python 3.14.7

```
        for rank, incident in enumerate(top_five_incidents, start=1):
            report_lines.append(
                f"{rank}. {incident['Country']} | "
                f"{incident['Attack Type']} | "
                f"{incident['Target Industry']} | "
                f"{incident['Financial Loss (in Million $)']:.2f} USD million"
            )
    else:
        report_lines.append("No matching incidents.")

    report_lines.append("\nOUTPUT FILE INFORMATION")
    report_lines.append("-" * 40)

    output_files = [
        FILTERED_FILE,
        TOP5_FILE,
        SUMMARY_FILE,
        BACKUP_FILE,
        REPORT_FILE
    ]

    for output_file in output_files:
        if output_file.exists():
            report_lines.append(
                f"{output_file.name}: {output_file.stat().st_size} bytes"
            )
        else:
            report_lines.append(f"{output_file.name}: Not yet created")

    report_lines.append("\nBACKUP VERIFICATION")
    report_lines.append("-" * 40)
    report_lines.append(f"Backup Exists: {backup_exists}")
    report_lines.append(f"Backup Verified: {backup_verified}")

    with open(REPORT_FILE, "w", encoding="utf-8") as file:
        file.write("\n".join(report_lines))

    print("Text Report Created:", REPORT_FILE.exists())
    print("\n===== CYBERSECURITY INCIDENT REPORT =====")
    print(REPORT_FILE.read_text(encoding="utf-8"))
```

[38] ✓ 0.0s Python

```
Text Report Created: True

===== CYBERSECURITY INCIDENT REPORT =====
CYBERSECURITY INCIDENT REPORT
========================================
Student: Correa
Student ID: TUPM-26-1106
Target Year: 2021

DATASET SUMMARY
```

Lab6_File_Handling_Correa main* ⊗ 0 ⚠ 0 Ln 16, Col 49 Spaces: 4 Spaces: 4 LF Cell 14 of 14

CORREA

Update

filing.ipynb U | Exercise3_Cybersecurity_Incident_Report.ipynb U | Global_Cybersecurity_Threats_2015-2024.csv U | user_behavior_dataset.csv U | weblog.csv U | household_power_consumption.txt .../Lab6_File_Handlir

Lab6_File_Handling_Correa > Lab6_File_Handling_Additional_Exercises > Exercise3_Cybersecurity_Incident_Report.ipynb > # [CELL 13] GENERATE CYBERSECURITY INCIDENT TEXT REPORT

Jupyter Notebook

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ...

Python 3.14.7

```
    print("Text Report Created:", REPORT_FILE.exists())
    print("\n===== CYBERSECURITY INCIDENT REPORT =====")
    print(REPORT_FILE.read_text(encoding="utf-8"))
```

[38] ✓ 0.0s Python

```
Text Report Created: True

===== CYBERSECURITY INCIDENT REPORT =====
CYBERSECURITY INCIDENT REPORT
========================================
Student: Correa
Student ID: TUPM-26-1106
Target Year: 2021

DATASET SUMMARY
----------------------------------------
Total Records: 3000
Incidents in Target Year: 299
Invalid Year Records: 0
Invalid Financial Records: 0

INCIDENT COUNT PER ATTACK TYPE
----------------------------------------
DDoS: 47
Malware: 49
Man-in-the-Middle: 48
Phishing: 41
Ransomware: 55
SQL Injection: 59

FINANCIAL LOSS SUMMARY
----------------------------------------
Total Financial Loss: 15873.41 USD million
Average Financial Loss: 53.09 USD million
```

```
    # [CELL 14] VERIFY ALL OUTPUT FILES

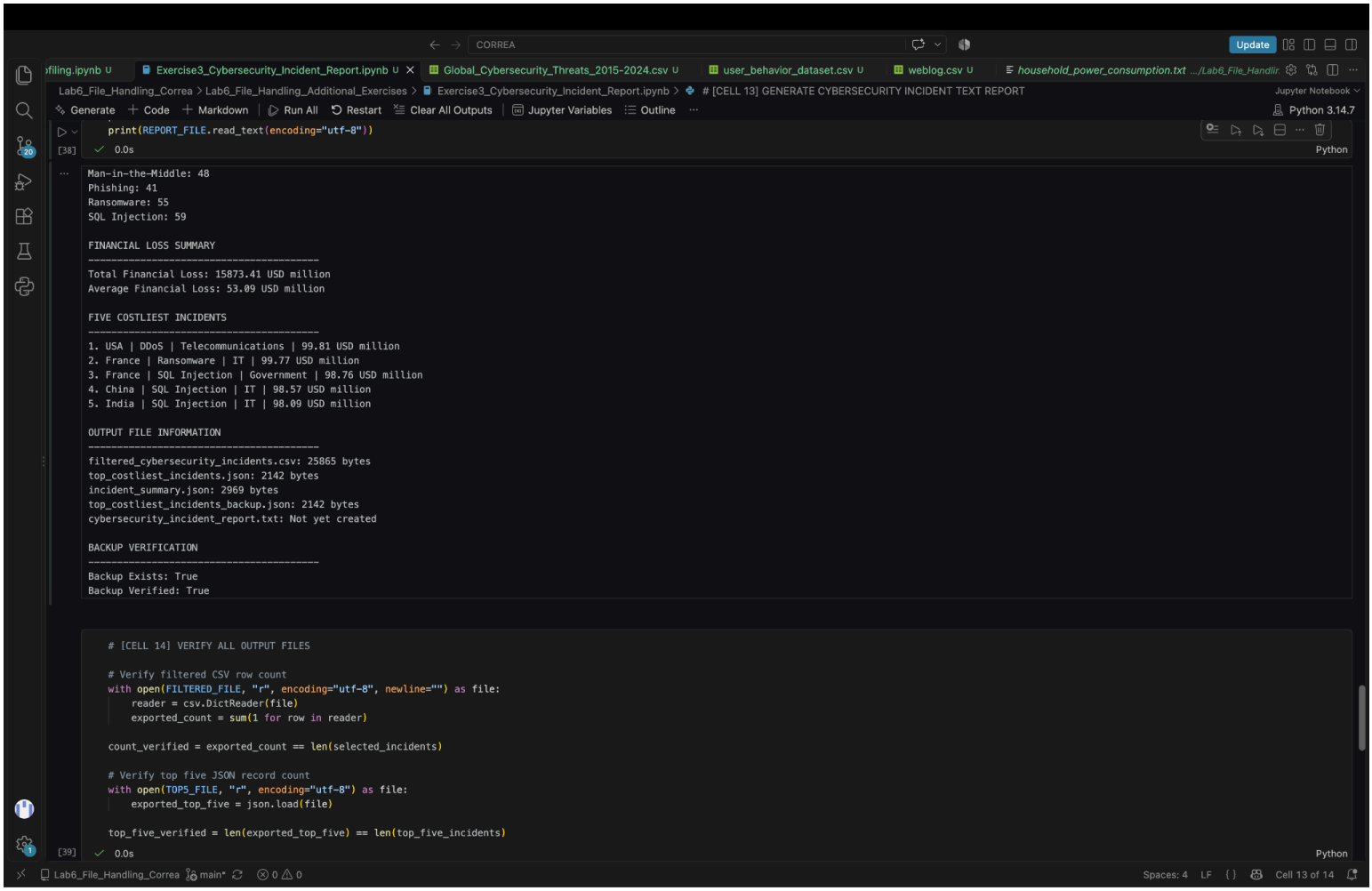
    # Verify filtered CSV row count
    with open(FILTERED_FILE, "r", encoding="utf-8", newline="") as file:
        reader = csv.DictReader(file)
        exported_count = sum(1 for row in reader)

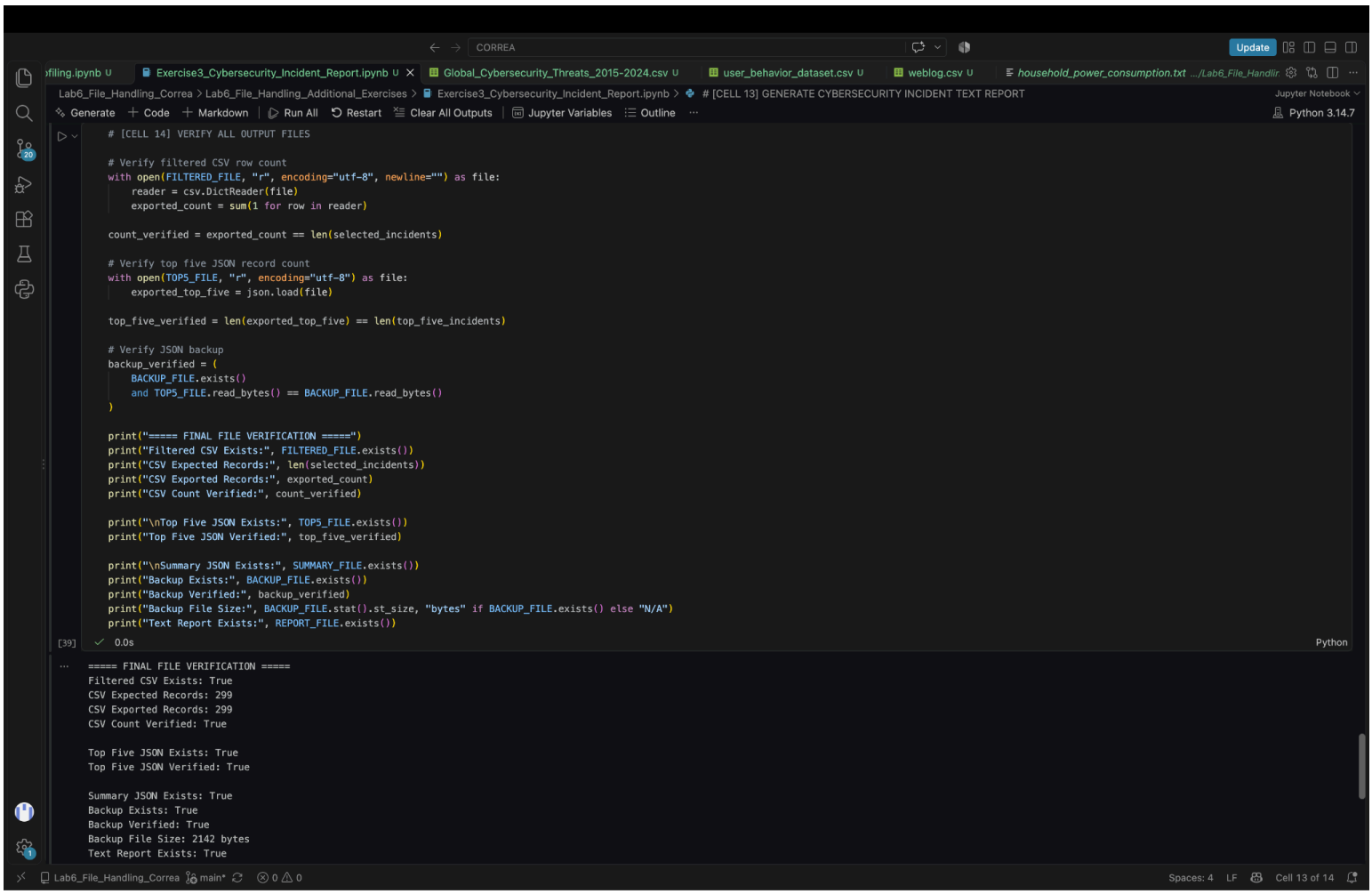
    count_verified = exported_count == len(selected_incidents)

    # Verify top five JSON record count
    with open(TOP5_FILE, "r", encoding="utf-8") as file:
        exported_top_five = json.load(file)
```

[39] ✓ 0.0s Python

Lab6_File_Handling_Correa main* ⊗ 0 ⚠ 0 Spaces: 4 LF { } Cell 13 of 14


CORREA
Update
filing.ipynb U
Exercise3_Cybersecurity_Incident_Report.ipynb U
Global_Cybersecurity_Threats_2015-2024.csv U
user_behavior_dataset.csv U
weblog.csv U
household_power_consumption.txt .../Lab6_File_Handlir
Lab6_File_Handling_Correa > Lab6_File_Handling_Additional_Exercises > Exercise3_Cybersecurity_Incident_Report.ipynb > # [CELL 13] GENERATE CYBERSECURITY INCIDENT TEXT REPORT
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.7
print(REPORT_FILE.read_text(encoding="utf-8"))
[38]
0.0s
Python
Man-in-the-Middle: 48
Phishing: 41
Ransomware: 55
SQL Injection: 59
FINANCIAL LOSS SUMMARY
Total Financial Loss: 15873.41 USD million
Average Financial Loss: 53.09 USD million
FIVE COSTLIEST INCIDENTS
1. USA | DDoS | Telecommunications | 99.81 USD million
2. France | Ransomware | IT | 99.77 USD million
3. France | SQL Injection | Government | 98.76 USD million
4. China | SQL Injection | IT | 98.57 USD million
5. India | SQL Injection | IT | 98.09 USD million
OUTPUT FILE INFORMATION
filtered_cybersecurity_incidents.csv: 25865 bytes
top_costliest_incidents.json: 2142 bytes
incident_summary.json: 2969 bytes
top_costliest_incidents_backup.json: 2142 bytes
cybersecurity_incident_report.txt: Not yet created
BACKUP VERIFICATION
Backup Exists: True
Backup Verified: True
[CELL 14] VERIFY ALL OUTPUT FILES
Verify filtered CSV row count
with open(FILTERED_FILE, "r", encoding="utf-8", newline="") as file:
reader = csv.DictReader(file)
exported_count = sum(1 for row in reader)
count_verified = exported_count == len(selected_incidents)
Verify top five JSON record count
with open(TOP5_FILE, "r", encoding="utf-8") as file:
exported_top_five = json.load(file)
top_five_verified = len(exported_top_five) == len(top_five_incidents)
[39]
0.0s
Python
Lab6_File_Handling_Correa main*
Spaces: 4
LF
Cell 13 of 14


CORREA
Update
filing.ipynb U
Exercise3_Cybersecurity_Incident_Report.ipynb U
Global_Cybersecurity_Threats_2015-2024.csv U
user_behavior_dataset.csv U
weblog.csv U
household_power_consumption.txt .../Lab6_File_Handlir
Lab6_File_Handling_Correa > Lab6_File_Handling_Additional_Exercises > Exercise3_Cybersecurity_Incident_Report.ipynb > # [CELL 13] GENERATE CYBERSECURITY INCIDENT TEXT REPORT
Jupyter Notebook
Generate
Code
Markdown
Run All
Restart
Clear All Outputs
Jupyter Variables
Outline
Python 3.14.7
[CELL 14] VERIFY ALL OUTPUT FILES
Verify filtered CSV row count
with open(FILTERED_FILE, "r", encoding="utf-8", newline="") as file:
reader = csv.DictReader(file)
exported_count = sum(1 for row in reader)
count_verified = exported_count == len(selected_incidents)
Verify top five JSON record count
with open(TOP5_FILE, "r", encoding="utf-8") as file:
exported_top_five = json.load(file)
top_five_verified = len(exported_top_five) == len(top_five_incidents)
Verify JSON backup
backup_verified = (
BACKUP_FILE.exists()
and TOP5_FILE.read_bytes() == BACKUP_FILE.read_bytes()
)
print("===== FINAL FILE VERIFICATION =====")
print("Filtered CSV Exists:", FILTERED_FILE.exists())
print("CSV Expected Records:", len(selected_incidents))
print("CSV Exported Records:", exported_count)
print("CSV Count Verified:", count_verified)
print("\nTop Five JSON Exists:", TOP5_FILE.exists())
print("Top Five JSON Verified:", top_five_verified)
print("\nSummary JSON Exists:", SUMMARY_FILE.exists())
print("Backup Exists:", BACKUP_FILE.exists())
print("Backup Verified:", backup_verified)
print("Backup File Size:", BACKUP_FILE.stat().st_size, "bytes" if BACKUP_FILE.exists() else "N/A")
print("Text Report Exists:", REPORT_FILE.exists())
[39]
0.0s
Python
===== FINAL FILE VERIFICATION =====
Filtered CSV Exists: True
CSV Expected Records: 299
CSV Exported Records: 299
CSV Count Verified: True
Top Five JSON Exists: True
Top Five JSON Verified: True
Summary JSON Exists: True
Backup Exists: True
Backup Verified: True
Backup File Size: 2142 bytes
Text Report Exists: True
Lab6_File_Handling_Correa main*
Spaces: 4
LF
Cell 13 of 14